Scout from the Inside

A beginner developer's guide to the Studies Weekly AI assistant

How the frontend (swo-3.0 / frontend-v3), the Laravel backend and the AI service (rag) work together — explained step by step, with diagrams and real code.

Edition: October 2026 · Code checked against branch ONLINE-5385_scout_release_2

How to read this book

Dear new team member, welcome! This book is written for you. You may be new to AI work, or new to this code base, or both. That is completely fine. Every chapter starts from the basics, uses simple English and shows real code from our two repositories.

- **Read chapters 1 to 3 first.** They give you the big picture and get your machine running.
- **Chapters 4 to 7** are about the browser side: how a page tells Scout what it shows, how Scout acts safely, and how we keep student names private.
- **Chapters 8 to 11** follow one chat message through the AI service (rag), the streaming lambda that serves it, and the Laravel backend.
- **Chapters 12 to 14** cover cost, testing, and a step-by-step recipe for your first feature.
- Every chapter ends with a **summary box**. If you are short on time, read the summaries first.

Every code block shows its file path at the top, like this:

```
swo-3.0/frontend-v3/ai/composables/useAiEnabled.ts
export function useAiEnabled() { ... }
```

Paths starting with `swo-3.0/` are in the main product repo. Paths starting with `rag/` are in the AI service repo. Many snippets are shortened so you can focus on the main idea. A `…` means some lines were left out. Open the real file to see everything — the comments there explain the reasoning behind each decision.

> **In simple words**
> Scout is our AI teaching assistant. A teacher chats with it inside Studies Weekly Online. Scout can answer questions about the page, find content, write lesson plans and questions, explain student results, and — with the teacher's permission — take actions in the app.

Contents

1. Welcome to Scout 4

2. The Big Picture 6

3. Setting Up Your Machine 8

4. A Tour of the Frontend AI Module 10

5. Surfaces: How a Page Tells Scout What It Shows 13

6. Capabilities and the Executor 17

7. Keeping Students Private 20

8. One Chat Turn, From Start to Finish 23

9. Inside the rag AI Service 26

10. Scout Core v2: The Streaming Lambda 31

11. Laravel's Part: Actions, Facts and Credits 37

12. Cost, Speed and Safety 44

13. Testing and Checking Your Work 51

14. Your First Feature: A Recipe 53

Chapter 1

Welcome to Scout

What we built, who it is for, and where the code lives.

1.1 What is Scout?

Scout is a chat assistant that lives inside Studies Weekly Online (SWO). A teacher opens it from the top bar and talks to it in normal language: "*Who is falling behind in my class?*", "*Make a lesson pack for week 5*", "*Level this article two grades down*". Scout answers using the teacher's real data and our real curriculum, not random internet text.

Three things make Scout different from a normal chatbot:

- **It knows the page.** Scout can see which article, class or report the teacher has open, so the teacher never has to copy and paste.
- **It can act, but only with permission.** Scout can create an assignment or move a student, but it always shows a confirmation card first. The teacher stays in charge.
- **It protects children.** Student names never go to the AI model. The model only sees codes like `Student K7QF2M`.

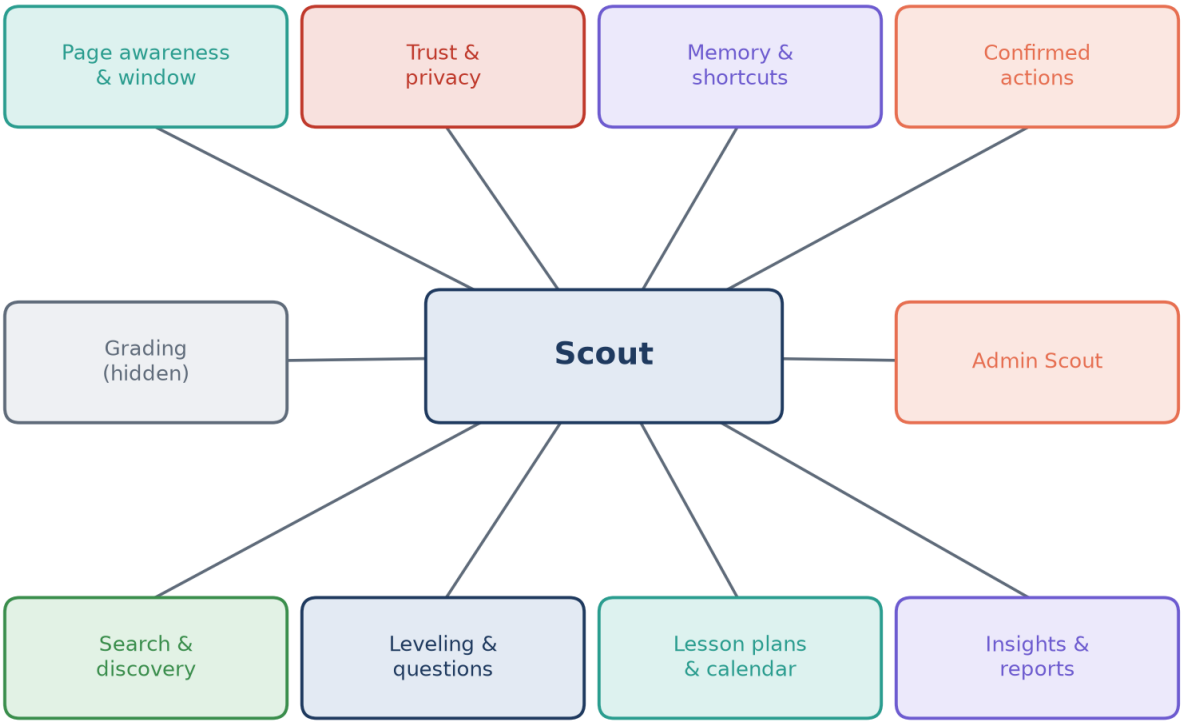


Figure 1.1 — The ten feature areas of Scout. Each one is a set of Jira tickets.

1.2 Where the work stands

The Scout workstream has 75 Jira tickets. Most of them are finished. Grading features are built and in production, but their buttons are hidden until a final QA round (ticket ONLINE-7865).

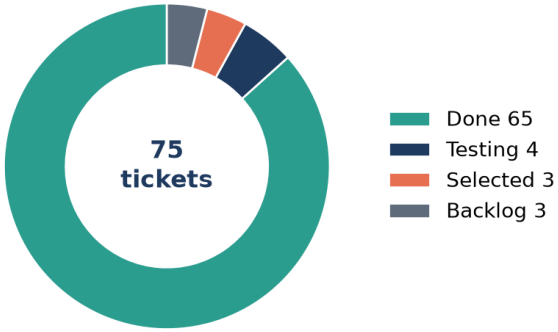


Figure 1.2 — Ticket status in October 2026.

1.3 The code bases

Folder	What it is	Language
`swo-3.0/frontend-v3`	The new teacher app (served at /beta/). Most Scout UI work happens here, in the `ai/` folder.	Nuxt 3, Vue 3, TypeScript, Bun
`swo-3.0/backend`	The main API: login, classes, content, plus Scout's confirmed actions, insights, credits and audit log.	Laravel 9, PHP 8.3
`rag/lambdas/ai_assistant`	The AI service. It talks to the model, runs tools and streams answers back.	Python, FastAPI, AWS Lambda
`swo-3.0/frontend`	The old Vue 2 app. Being replaced. Touch it only when you must.	Nuxt 2, Vue 2

Why it is built this way

The two repos are versioned separately but work as one system. An AI feature usually needs a change in **frontend-v3** and a change in **rag**. The agreement between them is written in `swo-3.0/docs/AiAssistantContract.md`, and the frontend is the source of truth for the data shapes sent over the wire.

Chapter summary

- ✓ Scout is an AI assistant inside SWO for teachers (and now district and school admins).
- ✓ It knows the page, it acts only after the teacher confirms, and it never sends student names to the model.
- ✓ Code lives in three places: frontend-v3 (browser), backend (Laravel) and rag (AI service).
- ✓ 65 of 75 tickets are done; grading is built but hidden until final QA.

Chapter 2

The Big Picture

How the browser, the AI service and Laravel talk to each other.

2.1 The architecture in one diagram

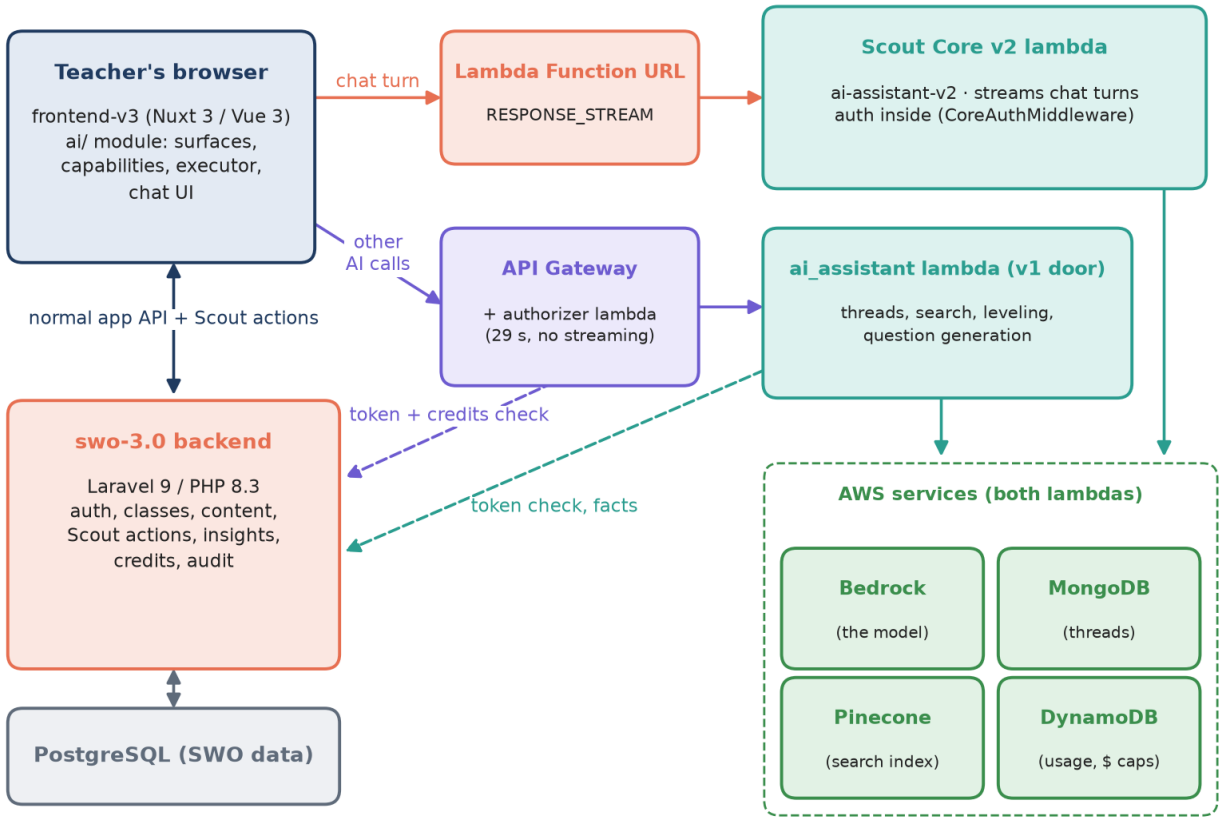


Figure 2.1 — The main parts of the system and how a request flows.

Let us walk through the diagram from left to right.

- **The teacher's browser** runs frontend-v3. Its `ai/` folder collects facts about the page, shows the chat window and runs actions.
- **The browser calls rag directly.** It is *not* proxied through Laravel. Every request carries the teacher's Laravel login token (JWT) in the `Authorization` header, plus `?host=` so rag knows which SWO site the teacher belongs to.
- **Chat turns go to Scout Core v2**, a separate lambda behind a **Lambda Function URL** that streams the reply word by word. API Gateway cannot stream, so the turn cannot go through it. This lambda checks the token itself (Chapter 10).
- **Everything else goes through API Gateway** to the original `ai_assistant` lambda: the thread list, search, renaming, article leveling and question generation. Its **authorizer lambda** checks the token and asks Laravel whether the teacher may use AI (`GET api/ai/user/credits`).
- **Both lambdas run the same rag code** (`lambdas/ai_assistant/src`). They call the model on AWS Bedrock, store threads in MongoDB, search content in Pinecone and track spending in DynamoDB.

- **Laravel** owns the school data. Scout asks it for facts (insights, pacing) and uses it to carry out confirmed actions.

> **In simple words**
> Think of a restaurant. The browser is the table where the customer (teacher) sits. The rag service is the kitchen that cooks the answer. Laravel is the store room that holds all the real ingredients (data). The chat turn goes through a special serving hatch (the Function URL) so food reaches the table bite by bite instead of all at once.

2.2 The five golden rules

These rules come from things that went wrong in the past. Please never break them.

Rule	What it means for you
1. Student names never reach the model	Use pseudonyms from `ai/context/roster.ts`. `publicFacts()` is an *allowlist*: you add the fields you want to share. Never copy a whole student object.
2. Publish answers, not rows	Every fact you put on the page card is sent on *every* turn and costs money. Send "12 of 28 students are below 60%", not 28 rows of scores. Models count badly.
3. Never cut data silently	If something does not fit, drop the whole section and *say so* in the text. A half list makes the model give a wrong answer with full confidence.
4. Measure against real data	Test with the local database, not only with made-up fixtures. Fixtures once passed while every article title was the literal word "Article".
5. Fix the words before the model	When Scout answers badly, the fix is almost never a bigger model. Usually it is a tool result's wording, an instruction in the wrong place, or an error message that says "wrong" without saying what is right.

2.3 One feature flag controls everything

All of Scout is behind one flag: `feature_frontend_ai_assistant`. If it is off, the teacher sees no Scout at all. Chapter 4 shows the one composable that reads this flag.

Chapter summary

- ✓ Chat turns stream from the Scout Core v2 lambda (Function URL); other AI calls go through API Gateway to the ai_assistant lambda.
- ✓ The browser talks to rag directly; Laravel handles credits, audit, facts and confirmed actions.
- ✓ Five golden rules: no student names to the model, publish answers not rows, never cut data silently, measure on real data, fix wording before blaming the model.
- ✓ One feature flag, feature_frontend_ai_assistant, switches the whole module on or off.

Chapter 3

Setting Up Your Machine

Run everything locally and avoid the traps that cost other people hours.

3.1 swo-3.0 runs in Docker

terminal

```
cd ~/swo-3.0
docker compose build && docker compose up        # profiles come from the root .env
COMPOSE_PROFILES=vue3 docker compose up          # only the Vue 3 app
# Vue 3 app:  https://online-v3.studiesw.test:3002/beta/   (the bare origin gives 404)
# API:        :3001
```

Follow `swo-3.0/README.md` for the hosts file, `.env` copies, `key:generate` and `config:clear`. Copying the env files alone does not give a working API.

3.2 frontend-v3 uses Bun

terminal

```
cd ~/swo-3.0/frontend-v3
bun install && bun run dev          # https on :3002
bun run build
```

3.3 rag runs on your host with uv

terminal

```
cd ~/rag
make install-dev     # one virtual env per lambda
make run             # starts local MongoDB, then services on :8000 :8001 :8002
make help
```

The Scout service you will work with most is `ai_assistant` on port `8002`. Locally, chat turns are served there too. To run the turn exactly the way the deployed streaming lambda runs it (Chapter 10):

terminal

```
cd ~/rag/lambdas/ai_assistant
uv run uvicorn src.core.lambda_stream:app --port 8080
```

3.4 Traps that cost hours

Trap	What to do
Each rag lambda reads its **own** .env	For ai_assistant it is `lambdas/ai_assistant/src/.env`. A .env in the repo root is read by nothing.
boto3 cannot see your .env	pydantic-settings does not export .env into the environment. Use `AWS_PROFILE` and `AWS_DEFAULT_REGION` (botocore ignores `AWS_REGION`).
Parameter Store overwrites your .env	`USE_PARAMETER_STORE` is true by default. Set `LOCAL_SERVER=true` to keep your local Mongo settings.

Trap	What to do
`/health` says healthy even when broken	Read the startup log. A broken config fails later with "You have not defined a default connection".
Local AI calls need remote credentials	Set `NUXT_ENV_AI_WEBSITE_HOST` and `NUXT_ENV_AI_LOGIN_*` in `frontend-v3/.env` and restart the dev server.
`php artisan test` wipes your database	There is no separate test DB. Run single tests with `--filter`. To recover: `php artisan migrate:fresh --seed`.
Config is cached in the API container	After any backend .env change run `php artisan config:clear`. Never run config:cache locally.

Watch out

`swo-3.0/start.sh` is old and destructive (it uninstalls node and overwrites env files). Do not run it. Use `docker compose`.

Chapter summary

- ✓ swo-3.0 runs in Docker; frontend-v3 uses Bun on :3002; rag runs on the host with uv (Scout on :8002).
- ✓ Each rag lambda has its own .env; use AWS_PROFILE / AWS_DEFAULT_REGION for boto3.
- ✓ artisan test wipes the local DB; use --filter and re-seed when needed.
- ✓ Run config:clear after backend .env edits, and never use start.sh.

Chapter 4

A Tour of the Frontend AI Module

The ai/ folder: what each part does and the rules for using it.

4.1 Everything lives in one folder

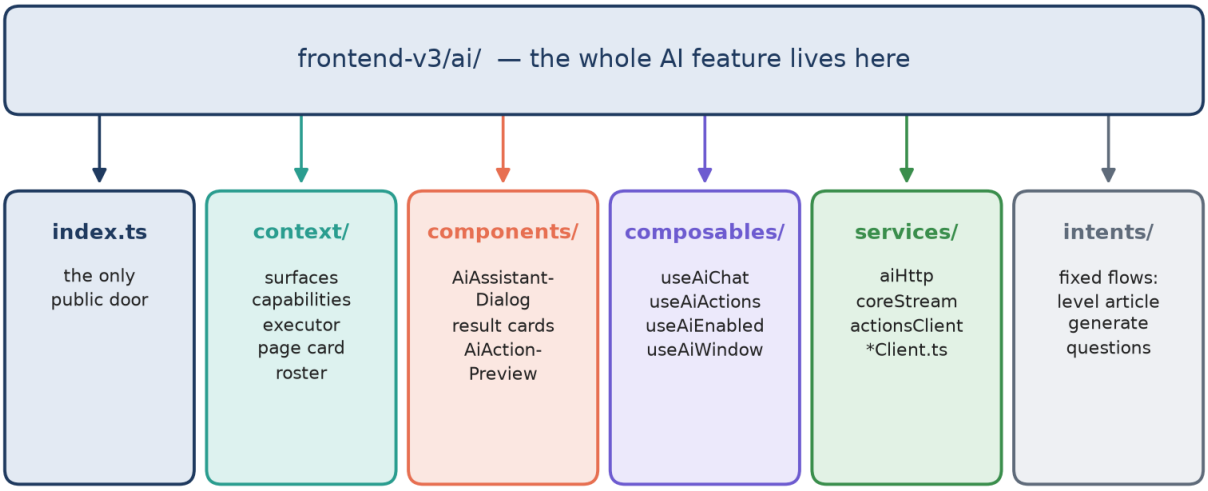


Figure 4.1 — The main folders inside frontend-v3/ai/.

The AI feature is a **self-contained module**. Code outside `ai/` may import only from `ai/index.ts`. This keeps the rest of the app clean, and it means the whole feature could be switched off or moved without breaking other pages.

swo-3.0/frontend-v3/ai/index.ts — *shortened*

```
/**
 * Public API of the AI feature module.
 *
 * Nothing outside `ai/` may import anything else from this folder.
 */
export { useAiAssistant } from './composables/useAiAssistant'
export { useAiEnabled, useAiAvailable, AI_FEATURE_FLAG } from './composables/useAiEnabled'
export { useAiTour, SCOUT_TOUR_NAME } from './composables/useAiTour'
export { default as AiAssistantRoot } from './components/AiAssistantRoot.vue'
export { default as AiLaunchButton } from './components/AiLaunchButton.vue'
```

4.2 The one feature-flag gate

No AI component may call `useFeatureFlag` directly. Everyone uses `useAiEnabled()`. Service code that runs outside a Vue setup function uses `assertAiEnabled()`, which **fails closed**: if the flag is not known yet, the answer is "off".

swo-3.0/frontend-v3/ai/composables/useAiEnabled.ts

```
export function useAiEnabled() {
    const { isFeatureEnabled } = useFeatureFlag()
    const store = useAiAssistantStore()
    const isAiEnabled = computed(() => {
        const enabled = isFeatureEnabled(AI_FEATURE_FLAG)
```

swo-3.0/frontend-v3/ai/composables/useAiEnabled.ts

```
        if (store.enabled !== enabled) {
            store.setEnabled(enabled)          // mirror it for code outside setup()
        }
        return enabled
    })
    return { isAiEnabled }
}

export function assertAiEnabled(): boolean {
    try {
        return useAiAssistantStore().enabled
    } catch {
        // Pinia not ready — fail closed.
        return false
    }
}
```

In simple words
"Fail closed" means: when in doubt, say no. Like a shop shutter that stays down until the owner is sure it is safe to open. No AI request can leave the browser before the flag is known.

4.3 Where are the AI services? (ai.config.ts)

There are no hard-coded URLs. One variable, `NUXT_ENV_AI_URL`, gives the base address, and each service sits at `<base>/<service>`. If it is missing, the first AI request throws an error that names the variable to set.

swo-3.0/frontend-v3/ai.config.ts — *shortened*

```
export type AiService = 'assistant' | 'level-content' | 'revise-question'
const BASE = 'NUXT_ENV_AI_URL'
/**
 *   aiUrl('assistant')                      → <base>/assistant
 *   aiUrl('assistant', 'thread', 'listing') → <base>/assistant/thread/listing
 */
export function aiUrl(service: AiService, ...path: string[]): string {
    const base = value(publicEnv()[BASE])
    if (!base) {
        throw new Error(`AI is not configured: set ${BASE}. See .env.example.`)
    }
    return [withoutTrailingSlash(base), service, ...path].join('/')
}
```

Chat turns use a second variable, `NUXT_ENV_AI_CORE_URL`: the Function URL of the streaming lambda. Leave it blank locally, where turns go to `:8002` like everything else. A deployed build that leaves it blank fails the turn with a message naming the variable (`coreRoot()` in `ai/services/coreClient.ts`).

Watch out
Values must come from `runtimeConfig.public`. In the browser, `process.env` is empty, so reading it there silently gives you nothing.

4.4 How the browser calls rag (aiHttp.ts)

`aiHttp.ts` is the **only** place that knows the browser talks to rag directly. Every request checks the flag, adds `?host=`, and sends the Laravel token as `Authorization`.

swo-3.0/frontend-v3/ai/services/aiHttp.ts — *shortened*

```
export async function aiFetch<T = any>(url: string, options: AiHttpOptions): Promise<T> {
    if (!assertAiEnabled()) {
        throw toRequestError({ message: AI_DISABLED_MESSAGE }, 403)
    }
    const { method = 'POST', body, auth, query, signal, timeoutMs } = options
    const params = new URLSearchParams({ host: auth.host })
    …
    response = await fetch(`${url}?${params.toString()}`, {
        method,
        headers: {
            'Content-Type': 'application/json',
            Authorization: auth.token,
        },
        body: body === undefined ? undefined : JSON.stringify(body),
        signal: effectiveSignal,
    })
    …
}
```

4.5 Two code-style rules to remember

- Only `~/components/global` auto-imports. Everything else must be imported or lazy-loaded.
- JS/Vue style: 4-space indent, camelCase functions, snake_case variables, `<script setup>` for new code, async/await instead of promise chains.

Chapter summary

- ✓ All AI frontend code is in frontend-v3/ai/; other code imports only from ai/index.ts.
- ✓ useAiEnabled() is the only flag gate; assertAiEnabled() fails closed.
- ✓ aiUrl() builds service URLs from NUXT_ENV_AI_URL; nothing is hard-coded.
- ✓ aiFetch() is the one place that sends the JWT and ?host= to rag.

Chapter 5

Surfaces: How a Page Tells Scout What It Shows

Components publish what they already know. No scraping of HTML.

5.1 The old way and the new way

The first version of Scout **scraped the page**: 161 special HTML attributes and a 933-line collector read the screen back out of the DOM. It broke often, and it could not see things that are not in the HTML at all (for example, reader mode lives in a Pinia store).

The new way turns this around. A Vue component already knows which tab is open and which article is loaded. So the component simply **publishes** that knowledge. We call each published bundle a **surface**.

> **In simple words**
> Old way: a stranger peeps through the window and guesses what is inside. New way: the shop owner puts up a clear signboard. The owner knows best, so the signboard is always right.

5.2 The shape of a surface

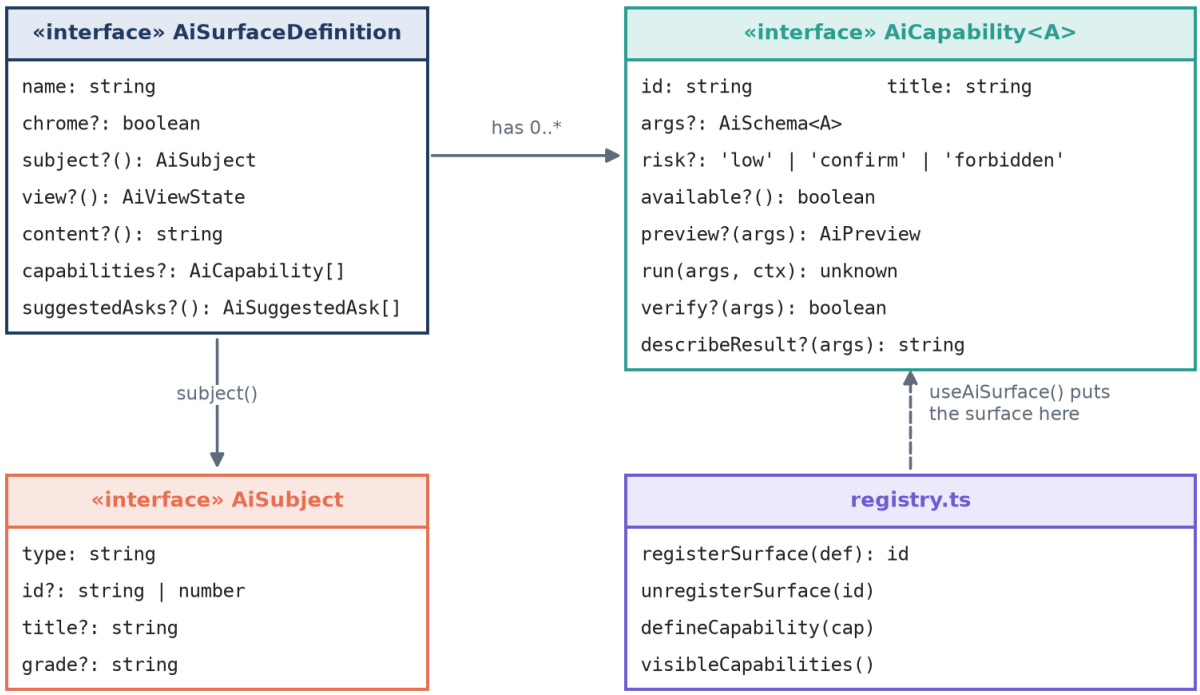


Figure 5.1 — UML view of a surface, its capabilities and the registry.

Field	Meaning	Example
`name`	A human name for the surface	`"Article rating"`

Field	Meaning	Example
`subject()`	What the page is about	`{ type: 'article', id: 42, title: 'Penguins' }`
`view()`	Small scalar facts (tab, filter, counts)	`{ articleRating: '4 of 5 stars' }`
`content()`	Anything list-shaped, already summarised	`"Unit 2 · Week 3: 4 articles…"`
`capabilities`	Things Scout can do here	`article.setRating`
`chrome`	true for navbar/sidebar furniture	`sorted last when space runs out`

5.3 useAiSurface(): registering a surface

A component calls `useAiSurface()` inside `<script setup>`. The composable registers the surface and watches `subject` and `view`. When they change, Scout's picture of the page is marked as changed. When the component unmounts, the surface is removed automatically.

swo-3.0/frontend-v3/ai/context/useAiSurface.ts — *comments removed*

```
export function useAiSurface(definition: AiSurfaceDefinition): AiSurfaceHandle {
    if (typeof window === 'undefined') return noop
    const stop = start(definition)
    let disposed = false
    const dispose = () => {
        if (disposed) return
        disposed = true
        stop()
    }
    if (getCurrentScope()) onScopeDispose(dispose)   // unmount = unregister
    return { dispose }
}

function start(definition: AiSurfaceDefinition): () => void {
    const tracked = [
        definition.subject ? () => safeRead(definition.subject!) : null,
        definition.view ? () => safeRead(definition.view!) : null,
    ].filter(Boolean) as Array<() => unknown>

    const id = registerSurface(definition)
    const stopTracked = tracked.length
        ? watch(tracked, () => markStateChanged(), { deep: true, flush: 'post' })
        : null
    return () => {
        stopTracked?.()
        unregisterSurface(id)
    }
}
```

5.4 A real, small example: rating an article

Here is a complete surface from the article page. Read it slowly — every Scout surface follows this pattern.

swo-3.0/frontend-v3/components/article/RateArticle.vue — *shortened*

```
useAiSurface({
    name: 'Article rating',
    view: () => {
        if (!isFeatureEnabled('feature_frontend_article_rating')) return {}
        if (loading.value) return { articleRating: 'loading' }
        return {
            // Spelled out so "not rated" cannot be read as a score of zero.
            articleRating: rating.value ? `${rating.value} of 5 stars` : 'not rated yet',
```

swo-3.0/frontend-v3/components/article/RateArticle.vue — *shortened*

```
            articleRatingEditable: !isAdoption.value,
        }
    },
    capabilities: [
        capability({
            id: 'article.setRating',
            panelLabel: 'Rate this article',
            title: 'Set the Article Rating stars — "rate this article 2", "give it 4 stars". …',
            group: 'Article tools',
            args: s.object({
                rating: s.number({ description: 'Stars, 1 to 5.', integer: true, min: 1, max: 5 }),
            }, { aliases: { stars: 'rating', value: 'rating', score: 'rating' } }),
            opensDialog: true,
            completesLater: true,          // nothing is saved until the teacher submits
            available: () => articleRatingOffered({ … }),
            run: (args) => showArticleRating({ index: starIndexFor(args.rating) }),
            verify: (args) => article_rating_dialog.value && rating.value === args.rating,
            describeResult: (args) =>
                `Opened the Rate Article dialog at ${args.rating} of 5 stars. Save it there.`,
        }),
    ],
})
```

Notice three things:

- `view()` writes "not rated yet" in words. A bare `0` could be read as "zero stars".
- `run()` calls the **same function the UI button calls**. Scout never clicks the DOM.
- `verify()` reads the app state back to check the dialog really opened with the right rating.

5.5 Surfaces stack like plates

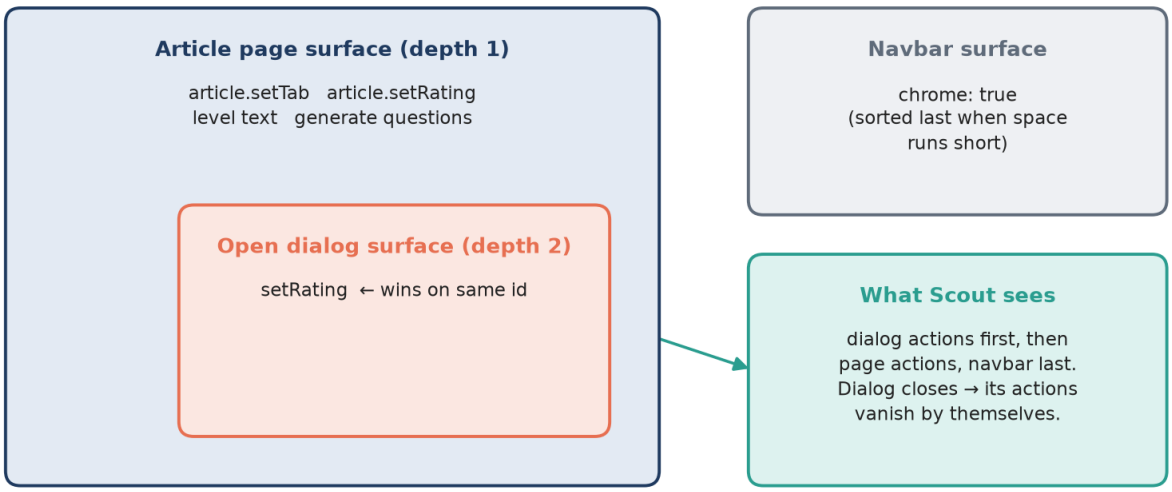


Figure 5.2 — An open dialog's surface sits on top of the page's surface.

Surfaces stack by mount depth. If two surfaces offer the same capability id, the **innermost wins**. So when a dialog is open, its actions shadow the page's actions, and when it closes they disappear by themselves — no clean-up code needed.

5.6 Budgets: keep the page card small

- The capability list sent to the model carries at most **32 entries**; extra entries become `…(+N more)`. That is why `chrome: true` surfaces sort last.
- Each line is capped at 480 characters, and the argument signature is kept at the end of the line.
- The whole serialised card is capped at 14,000 characters. If it is cut, the JSON breaks. Never raise one cap without the other.
- To see exactly what Scout sees, run `window.__aiContextDebug = true` in the browser console.

> **Try it yourself**
> Open any teacher page with the dev server running, set `window.__aiContextDebug = true`, open Scout and send a message. Look at the logged card. Is every number something a teacher would understand? Is anything a list that should be a summary?

Chapter summary

- ✓ A surface is what a component publishes about itself: subject, view, content and capabilities.
- ✓ useAiSurface() registers it, watches it with Vue watchers, and removes it on unmount.
- ✓ run() calls the app's own functions; verify() re-reads state to prove it worked.
- ✓ Surfaces stack; the innermost (for example an open dialog) wins on the same id.
- ✓ Keep the card small and honest: summaries not rows, and never cut silently.

Chapter 6

Capabilities and the Executor

How Scout changes things in the app — safely, and only when it really worked.

6.1 What is a capability?

A **capability** is one action Scout may take, like `article.setTab` or `students.move`. It is a typed call into the app's own stores. It is **never** a DOM click.

Because a capability uses the same store action as the UI, it hits the same API and the same permission checks. So Scout **cannot do anything the teacher could not do** themselves.

Risk tiers

risk	What happens	Example
`'low'`	Runs straight away (opening a tab, filtering a list).	`article.setTab`
`'confirm'`	Shows a confirmation card. Runs only after the teacher approves.	`assignment.assign`
`'forbidden'`	Scout can point to it, but will never do it.	`assignments.delete`

6.2 The executor: one gate for every action

All actions go through one file: `ai/context/executor.ts`. Its comment lists every gate, and each gate exists because something went wrong without it.



Figure 6.1 — The checks every action passes, in order.

`swo-3.0/frontend-v3/ai/context/executor.ts` — *header comment, shortened*

```
/**
 * The action executor: the single path through which the assistant may change anything.
 *
 *  1. Allowlist.  A capability the model invents is an error, not a click.
 *  2. Mode.       Plan mode executes nothing, enforced here rather than asked for
 *                 in a prompt the model could talk itself out of.
 *  3. Schema.     Model output is untrusted input and is validated as such.
 *  4. Risk tier.  Checked here, in code, rather than asserted in a system prompt.
 *  5. Staleness.  The page must still be the page the model planned against.
```

swo-3.0/frontend-v3/ai/context/executor.ts — *header comment, shortened*

```
 *  6. Verify.     Success is reported because app state says so, not because
 *                 `run()` resolved without throwing.
 */
```

In simple words
The executor is like the chowkidar at the school gate. It does not care how confident the visitor sounds. It checks the list, checks the ID card, and only then opens the gate. And after the visitor goes in, it checks the register to be sure they really arrived.

6.3 verify() is where reliability comes from

A `run()` that finishes without an error proves nothing. The store may still be loading, or a watcher may not have run yet. So the executor asks `verify()` a few times, with short waits.

swo-3.0/frontend-v3/ai/context/executor.ts — *two excerpts*

```
const config: ExecutorConfig = {
    timeoutMs: 10_000,              // ceiling on a single run()
    verifyBackoffMs: [0, 30, 120],  // wait before each verify() attempt
    dialogWaitMs: 5_000,            // longest wait for a dialog to appear
}
async function verifyWithRetries(capability: AiCapability<any>, args: unknown): Promise<boolean> {
    await nextTick()
    for (const delay of config.verifyBackoffMs) {
        if (delay > 0) await sleep(delay)
        try {
            if (capability.verify!(args) === true) return true
        } catch {
            // A throwing verify is a failed verify, not a crash.
        }
    }
    return false
}
```

In simple words
Like the DRS review in cricket: the umpire's first call is not final. We look again at the replay (the real app state) before we say "OUT" — or here, "Done".

Watch out
A capability without a real `verify()` must only report what it knows, for example "Opened the dialog — save it there". Never write success text for something you did not check.

6.4 Where capabilities live

- **Global capabilities** (available on every page, like navigation) are registered once with `defineCapability()` in `ai/context/registry.ts`.
- **Surface capabilities** live inside the component's `useAiSurface()` call, because they need the component's own handlers (for example `article.setTab` cannot be a free function).
- **Server-backed actions** (assign, move students, save a calendar) set `serverAction` and go through Laravel's confirm flow — see Chapter 11.

Chapter summary

- ✓ A capability is a typed call into the app's stores; Scout never clicks the DOM.
- ✓ Risk tiers: low runs at once, confirm needs the teacher's approval, forbidden is never run.
- ✓ executor.ts checks allowlist, mode, schema, risk and staleness before run().
- ✓ verify() re-reads app state with short retries; success is only reported when state proves it.

Chapter 7

Keeping Students Private

Pseudonyms in, real names back out — and only in the browser.

7.1 The rule

No student name, username or password ever leaves the browser. Pages about people publish **pseudonyms** instead, and the browser puts the real names back only when it shows the reply.

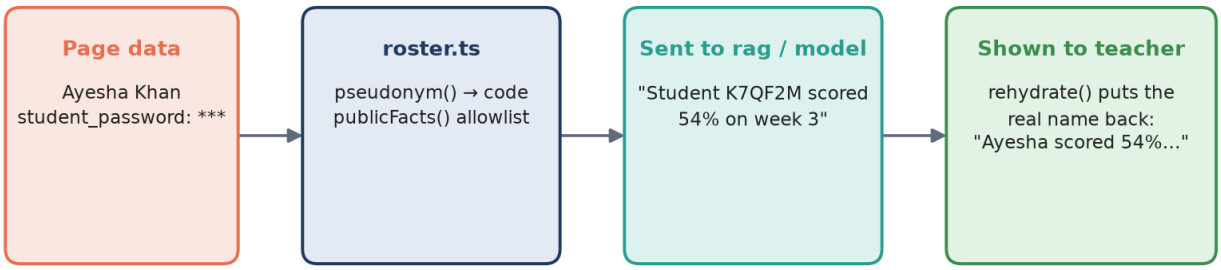


Figure 7.1 — A student's name turns into a code before anything is sent.

7.2 How a code is made

Each student gets a short code made with an HMAC (a keyed hash) of their id. The key is per user, so the same student gets the **same code** in every tab and every reopened conversation (ticket ONLINE-7535), but nobody can turn the code back into a name without the key.

swo-3.0/frontend-v3/ai/context/roster.ts

```
function codeFor(kind: 'Student' | 'Teacher', id: string): string {
    const digest = hmac(sha256, aliasKey!, new TextEncoder().encode(`${kind === 'Student' ? 's' : 't'}:${id}`))
    let n = 0n
    for (let index = 0; index < 16; index++) n = (n << 8n) | BigInt(digest[index])
    let code = CODE_FIRST[Number(n % BigInt(CODE_FIRST.length))]
    n /= BigInt(CODE_FIRST.length)
    for (let index = 1; index < CODE_LENGTH; index++) {
        code += CODE_REST[Number(n % BigInt(CODE_REST.length))]
        n /= BigInt(CODE_REST.length)
    }
    return code
}
export function pseudonym(student: any): string {
    return aliasFor(student, 'Student')        // e.g. "Student K7QF2M"
}
```

7.3 publicFacts(): an allowlist, not a filter

Student records in SWO carry fields like `student_password`. If you build the shared object by *removing* bad fields, one spread operator ({...student}) later will leak them. So `publicFacts()` starts empty and **adds** only safe fields, one by one.

swo-3.0/frontend-v3/ai/context/roster.ts

```
export function publicFacts(student: any): PublicStudentFacts {
    const facts: PublicStudentFacts = { student: pseudonym(student) }
    // Only settings that are genuinely set are published.
    if (typeof student?.accessibility_enabled === 'boolean') facts.accessibility = student.accessibility_enabled
    if (typeof student?.google_translate === 'boolean') facts.translate = student.google_translate
    if (typeof student?.crossword_word_bank === 'boolean') facts.wordBank = student.crossword_word_bank
    if (typeof student?.audio_reader === 'boolean') facts.audioReader = student.audio_reader
    if (typeof student?.show_vocabulary === 'boolean') facts.vocabulary = student.show_vocabulary
    return facts
}
```

7.4 rehydrate(): putting names back for the teacher

swo-3.0/frontend-v3/ai/context/roster.ts — *shortened*

```
export function rehydrate(text: string): string {
    aliasNamingVersion()                // re-render when the key or roster arrives
    if (!text) return text
    text = revealOwnDetails(text)
    if (!namedByCode.size) return text
    return text.replace(aliasInText(), (whole, prefix, code, offset, subject) => {
        const person = isAliasInText(prefix, code) ? namedByCode.get(code.toLowerCase()) : null
        if (!person || isMentioned(subject, offset, whole.length)) return whole
        return displayName(person)
    })
}
```

7.5 A second safety net in rag

Even if a name slips into the request (for example the teacher typed it), the rag service scrubs it **before reading, storing or sending anything**. The browser sends a list of names to scrub, and the service replaces them in every field. Teachers can also type `@` to pick a student, so the name never has to be guessed from free text (ONLINE-7565).

rag/lambdas/ai_assistant/src/routers/core_router.py — *inside turn()*

```
# R111 D2: names out of every browser-supplied field before anything reads,
# stores or sends it; the same scrub stays active as the backstop for every
# Bedrock request this turn builds.
listed = request.scrub_names is not None
scrub = NameScrub.from_wire(request.scrub_names)
request.scrub_names = None
if scrub is not None:
    scrub_chat_request(request, scrub)
activate(scrub)
```

After the model answers, more **screens** check the reply (Chapter 9): `name_claim.py` stops sentences that claim a hidden name, and `student_wording.py` keeps wording about children fair and evidence-based.

Chapter summary

- ✓ Student names never leave the browser; the model sees stable codes like Student K7QF2M.
- ✓ Codes come from an HMAC of the student id with a per-user key, so they are stable but not reversible.
- ✓ publicFacts() is an allowlist: start empty and add only safe fields.
- ✓ rehydrate() puts real names back only when showing the reply.
- ✓ rag scrubs names again on arrival, and screens check every reply.

Chapter 8

One Chat Turn, From Start to Finish

Follow a single message through the whole system.

8.1 The sequence diagram

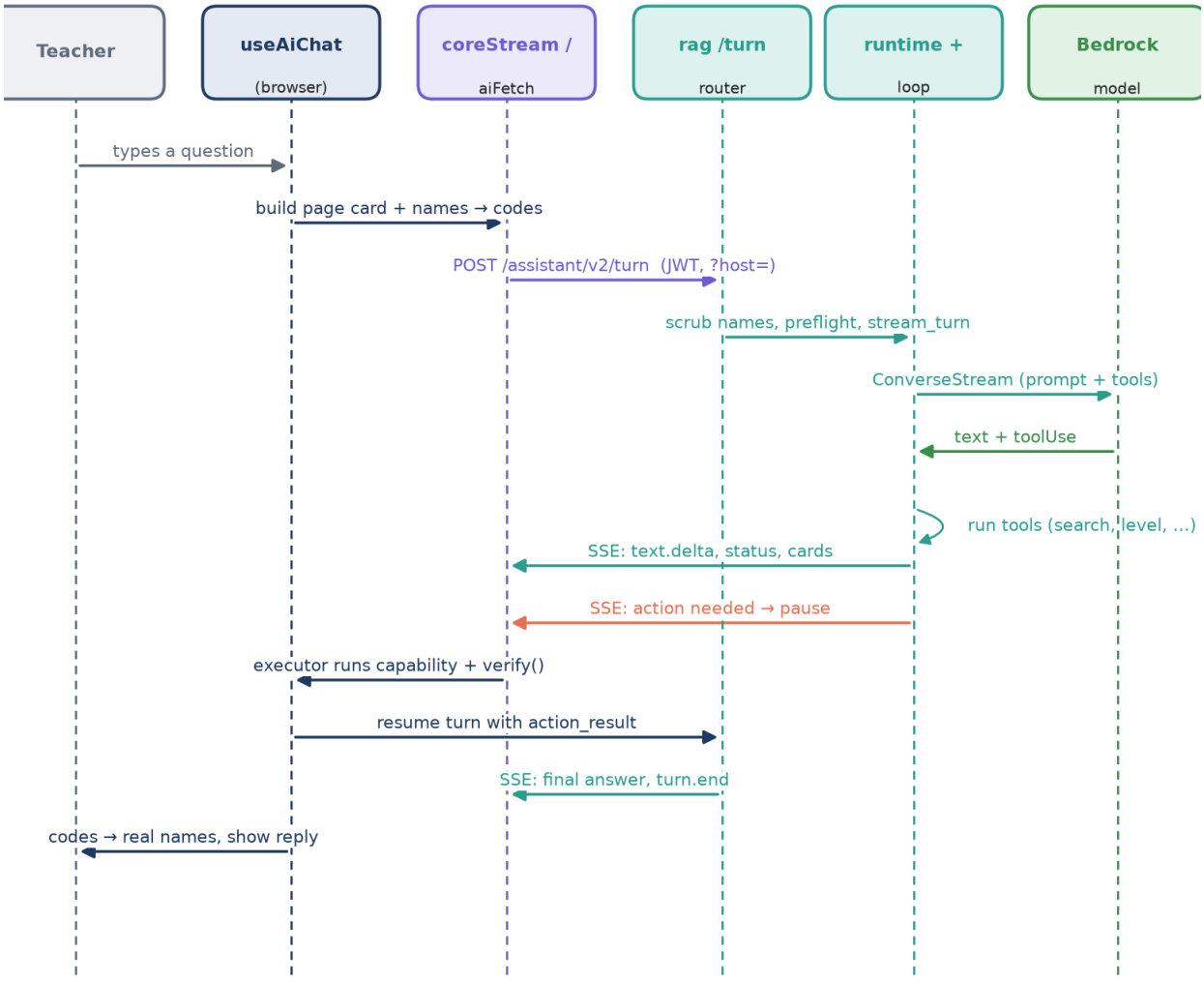


Figure 8.1 — UML sequence diagram of one chat turn, including a paused action.

8.2 Step by step

- **1. The teacher types.** `useAiChat.ts` takes the message.
- **2. The browser builds the page card.** `ai/context/pageCard.ts` (`buildPageCard`) collects all active surfaces, turns names into codes and keeps the card within budget.
- **3. The request goes out.** `ai/services/coreClient.ts` posts to `<NUXT_ENV_AI_CORE_URL>/v2/turn` (the streaming lambda's Function URL) with the JWT and `?host=`.
- **4. rag checks everything first.** `CoreAuthMiddleware` checks the caller (Chapter 10), then `core_router.turn()` scrubs names, checks sizes and identity, then `preflight()` checks the thread and the spending cap. Any problem here comes back as normal JSON with a clear

message.

- **5. The stream starts.** Only a turn that passed all checks gets `200 text/event-stream` (Server-Sent Events, SSE).
- **6. The agent loop runs.** The model streams text and tool calls; tools run; results go back to the model; this repeats until the model is done.
- **7. If an action is needed, the turn pauses.** The service sends the action to the browser. The executor runs it (with a confirm card if needed), verifies it, and sends the result back to resume.
- **8. The reply is shown.** Codes are turned back into names, and result cards (articles, lesson plans, charts) are drawn.

8.3 The /turn endpoint

rag/lambdas/ai_assistant/src/routers/core_router.py — *shortened*

```
@router.post("/turn")
async def turn(request: ChatRequest, http_request: Request, host: Optional[str] = None):
    started = time.monotonic()
    …                                                # name scrub (see 7.5)
    limits = get_core_config()
    if not (request.message or "").strip() and not request.action_result and compose is None:
        return json_error(400, "empty_message", "Message cannot be empty.")
    if len(request.message or "") > limits.max_message_chars:
        return json_error(413, "message_too_long",
                          f"That message is too long. Please keep it under "
                          f"{limits.max_message_chars:,} characters.")
    user_id = caller_id(request.user_id, http_request)
    if not user_id:
        return json_error(401, "unauthenticated", "Please sign in again.")
    from src.core.runtime import PreflightError, preflight, stream_turn
    try:
        prepared = await preflight(request, user_id)
    except PreflightError as failed:
        return JSONResponse(status_code=failed.status, content=failed.body())
    auth = scout_auth(http_request)
    await fill_local_teacher_context(request, http_request, auth)
    return StreamingResponse(stream_turn(request, user_id, prepared, started=started, auth=auth),
                             media_type="text/event-stream", headers=SSE_HEADERS)
```

Why it is built this way
Everything that can fail *before* the model runs is answered as ordinary JSON. That way the browser's normal error handling works, and a teacher sees "Please sign in again" instead of a broken stream.

8.4 Other doors on the same service

Route	Used for
`POST /turn`	Normal chat turns (teachers)
`POST /admin/turn`	District and school admin questions (ONLINE-7806)
`POST /grade, /group`	Grading suggestions and grouping similar answers
`POST /misconceptions`	Which wrong answers students chose, and why
`POST /grade-drawings`	Drawing and handwriting suggestions
`POST /read-calendar`	Reading an imported school calendar

Route	Used for
`POST /standards/suggest`	Suggesting standards for a teacher's own questions
`POST /generate-from-topic`	The question generator in Customized Content

Chapter summary

- ✓ Browser builds a page card (pseudonyms, budgets) and posts it to /assistant/v2/turn.
- ✓ rag validates everything first and replies with plain JSON on any early failure.
- ✓ Valid turns stream back as SSE: text, status labels, cards, and pause events for actions.
- ✓ Actions run in the browser, are verified, and the result resumes the turn.

Chapter 9

Inside the rag AI Service

Routers, the runtime, the agent loop, tools, engines, prompts and screens.

9.1 The map

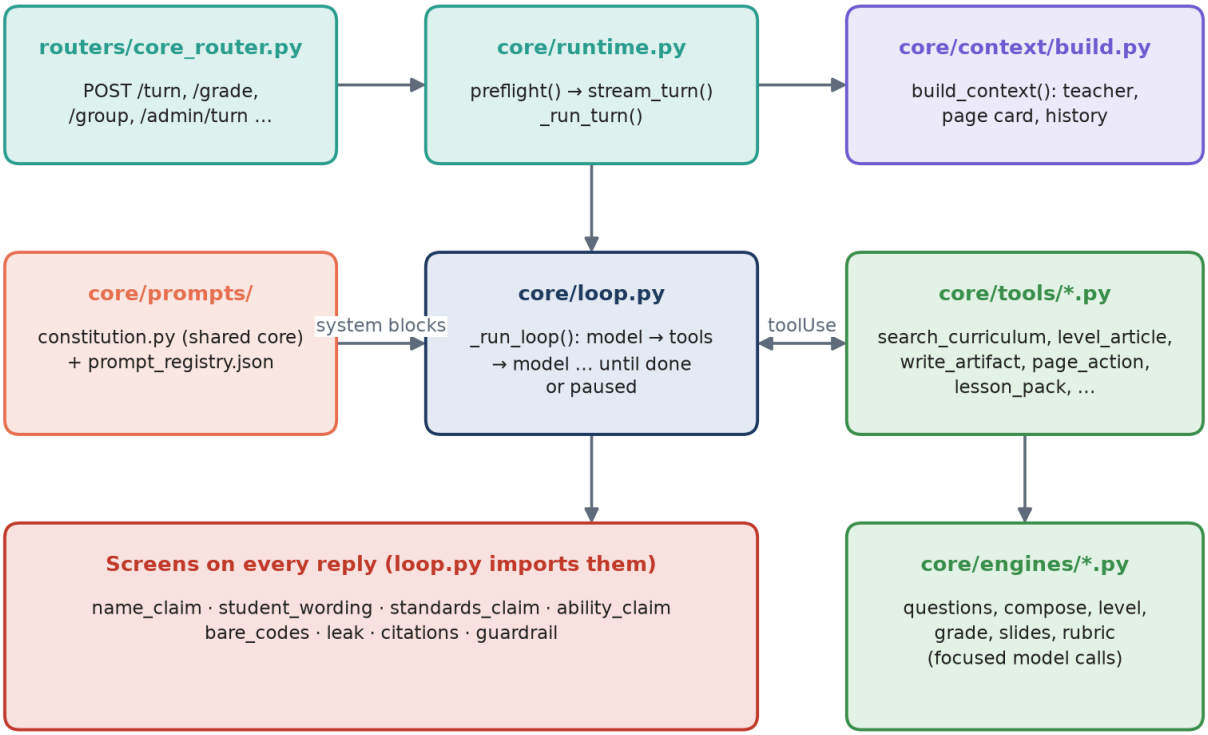


Figure 9.1 — The main parts of lambdas/ai_assistant/src/core.

Part	Job	Main file(s)
Router	HTTP doors, early checks	`routers/core_router.py`
Runtime	Prepares the turn, builds context, saves the thread, streams events	`core/runtime.py`
Context	Teacher block, page card, history	`core/context/build.py`, `teacher.py`
Loop	Model → tools → model until done or paused	`core/loop.py`
Tools	What the model may call (search, level, write…)	`core/tools/*.py`
Engines	Focused model calls for one job (questions, slides…)	`core/engines/*.py`
Prompts	Shared core rules + each feature's rules	`core/prompts/`, `prompt_registry.json`
Screens	Check every reply before it is sent	`name_claim.py, leak.py, …`

9.2 The agent loop

The loop is the heart of the service. Its job is simple to describe:

rag/lambdas/ai_assistant/src/core/loop.py — *module docstring*

```
"""The Scout Core v2 agent loop, on Bedrock ConverseStream.

    model streams → text.delta out → tool calls run in parallel → results in → repeat
    until the model ends its turn, or a tool pauses it for the browser.

No turn deadline: a model call is abandoned only when its stream goes idle, and
retried once if nothing had been shown yet. A tool that runs long gets `status`
heartbeats. The per-turn ceiling is enforced by the runtime, not here.
"""
```

In simple words

Imagine a dhaba waiter (the model) taking an order. If the customer asks for something the kitchen must prepare, the waiter goes to the kitchen (a tool), brings it back, and then continues talking. If the customer must decide something, the waiter pauses and waits.

9.3 Tools: the shape every tool follows

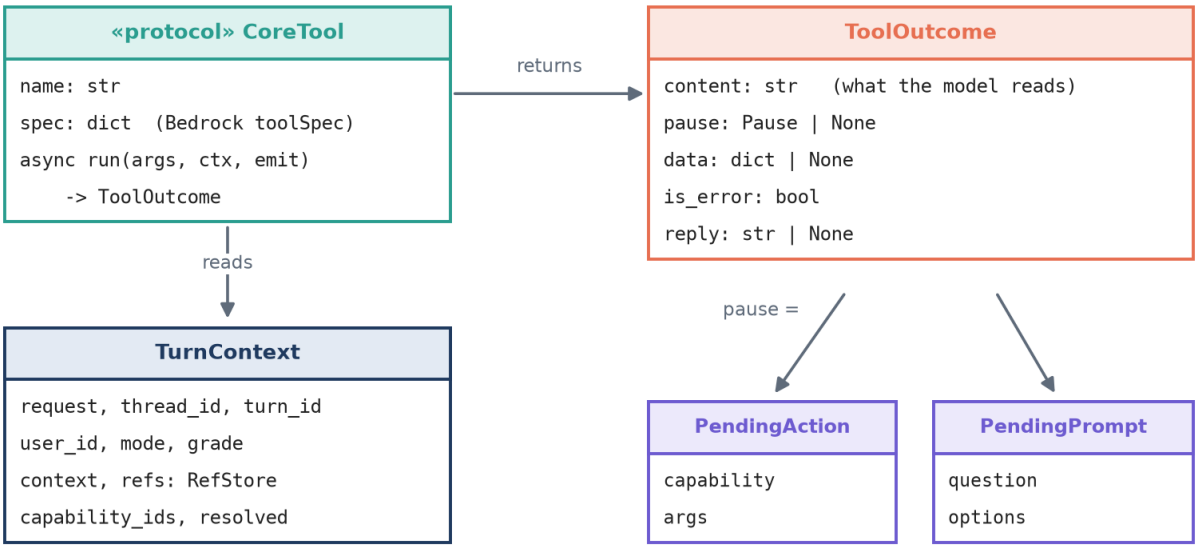


Figure 9.2 — UML view of the tool interface and what a tool returns.

rag/lambdas/ai_assistant/src/core/tools/base.py — *shortened*

```
"""The one interface every Scout Core v2 tool implements.

    class CoreTool(Protocol):
        name: str
        spec: dict                    # Bedrock toolSpec (name, description, inputSchema)
        async def run(self, args, ctx, emit) -> ToolOutcome: ...
"""
#: Streams one SSE event: `await emit("status", {"label": "Reading the article"})`.
Emit = Callable[[str, Dict[str, Any]], Awaitable[None]]
```

rag/lambdas/ai_assistant/src/core/tools/base.py — *shortened*

```
@dataclass
class TurnContext:
    """Everything a tool may read about the turn it runs in."""
    request: Any
    thread_id: str
    turn_id: str
    user_id: str
    mode: str
    publication_ids: List[str]
    grade: Optional[str]
    context: Any
    refs: RefStore
    capability_ids: List[str] = field(default_factory=list)
    …
```

A tool returns a `ToolOutcome`. Its `content` is what the **model** reads next. If the tool needs the teacher, it sets `pause` to a `PendingAction` (run a capability in the browser) or a `PendingPrompt` (ask a question with buttons).

Example: the forget tool asks before deleting everything

rag/lambdas/ai_assistant/src/core/tools/forget.py — *shortened*

```
CONFIRM_ALL = "Yes, forget everything"
KEEP_ALL = "No, keep my memory"
CONFIRM_ALL_QUESTION = "Forget everything Scout remembers about you? This can't be undone."
def confirm_all(ctx: TurnContext) -> ToolOutcome:
    """Pause on the confirmation. The two labels run `forget` again directly,
    so the click cannot be misread."""
    resume = {"tool": NAME, "args": {"memory_id": "all"}, "field": "confirm",
              "values": {CONFIRM_ALL: "yes", KEEP_ALL: "no"}}
    return ToolOutcome(
        content=ASKED_TO_CONFIRM,
        pause=PendingPrompt(id=ctx.tool_use_id or "", question=CONFIRM_ALL_QUESTION,
                            options=[CONFIRM_ALL, KEEP_ALL], allow_free_text=False,
                            resume=resume, resume_instruction=CONFIRM_RESUME),
    )
def forgotten_reply(texts: list, failed: int) -> str:
    """The teacher's reply, built from the rows that were really deleted,
    never from the model's account of them."""
    …
```

> **Why it is built this way**
> Look at `forgotten_reply`: the reply is built **in code** from what was really deleted. We do not let the model describe it, because a model can say "done" when it was not. This is the same idea as `verify()` in the browser.

9.4 Tools versus engines

A **tool** is something the chat model chooses to call. An **engine** is a focused model call that does one job well, with its own prompt — for example writing questions (`engines/questions.py`), composing a lesson plan (`engines/compose.py`) or drafting slides (`engines/slides.py`). Tools often call engines. Some routes (like `/grade`) call an engine directly with no chat model at all.

Tool (core/tools/)	What it does
`search_curriculum.py`	Finds articles, resources and media the teacher can open
`level_article.py`	Rewrites an article at a lower grade; asks for the grade if unknown

Tool (core/tools/)	What it does
`generate_questions.py`	Writes questions from an article
`write_artifact.py`	Writes lesson plans, rubrics and other documents
`lesson_pack.py`	Plan + worksheet + exit ticket + slides in one request
`page_action.py`	Asks the browser to run a page capability (pauses the turn)
`ask_user.py`	Asks the teacher a question with buttons (pauses the turn)
`remember.py / forget.py`	Saves or deletes what Scout remembers about the teacher

9.5 Prompts: a shared core and a registry

Every chat call sends a **constitution** (the shared system prompt). It is kept byte-stable so it can be cached. Per-turn things (date, page, mode) go in separate context blocks, never in the constitution.

`rag/lambdas/ai_assistant/src/core/prompts/constitution.py` — *module docstring*

```
"""The Scout Core v2 constitution.

One teacher-centred system prompt. It describes who Scout serves and how Scout
works; it is not a list of fixes. A change here must move a teacher-eval number,
and a failure is answered by the eval, not by adding a sentence.
The text is byte-stable: it sits before the first `cachePoint`, so anything that
varies per turn (mode, date, page) belongs in the context blocks, never here.
Every sentence here has an entry in `core/prompt_registry.json`
(`tests/core/check_prompt_registry.py` fails without one).
"""
```

The **prompt registry** gives every rule sentence exactly one home. A check script fails if you add a sentence without registering it. This stops the prompt from growing forever (ONLINE-7588 cut it from 17.7k to about 5.2k shared tokens).

9.6 Screens: checking the reply

After the model writes, small **screens** check the text before the teacher sees it. Each one looks for one kind of mistake:

Screen	Catches
`name_claim.py`	Sentences that claim to know a hidden student name
`student_wording.py`	Unfair or unsupported wording about a child
`standards_claim.py`	Claims that standards were added or placed when no tool did it
`ability_claim.py`	Scout describing abilities it does not have
`bare_codes.py`	Bare standard codes that should be explained
`leak.py`	Internal text (rules, tool output) leaking into the reply
`citations.py`	Links: a link survives only if a tool returned it this turn
`guardrail.py`	Harmful content, using the AWS Bedrock guardrail

Why it is built this way

The "invented URL" problem was solved by `citations.py`, not by a bigger model: a link can appear in a reply only if it came from a tool result in the same turn. Prompting alone never fixed it.

Chapter summary

- ✓ Router → runtime (preflight, context) → loop (model ⇄ tools) → screens → SSE to the browser.
- ✓ Every tool implements run(args, ctx, emit) and returns a ToolOutcome; pause hands control to the browser.
- ✓ Engines are focused model calls for one job; tools often call them.
- ✓ The constitution is byte-stable for caching; every rule sentence is registered in prompt_registry.json.
- ✓ Screens check every reply; facts like 'deleted' or links come from code, not from the model's words.

Chapter 10

Scout Core v2: The Streaming Lambda

Where the chat turn actually runs in AWS, and why it has its own lambda.

10.1 Why a second lambda?

Scout writes its answer **word by word**, like a person typing. For that, the server must send the reply as a stream (Server-Sent Events). But the API Gateway in front of our other AI lambdas is an HTTP API with a **29-second limit**, and it **cannot stream**. A lesson pack can take much longer than 29 seconds.

So the chat turn runs in its own lambda, **Scout Core v2**, reached through a **Lambda Function URL** with `invoke_mode = RESPONSE_STREAM`. It runs the same rag code you saw in Chapter 9, packaged differently.

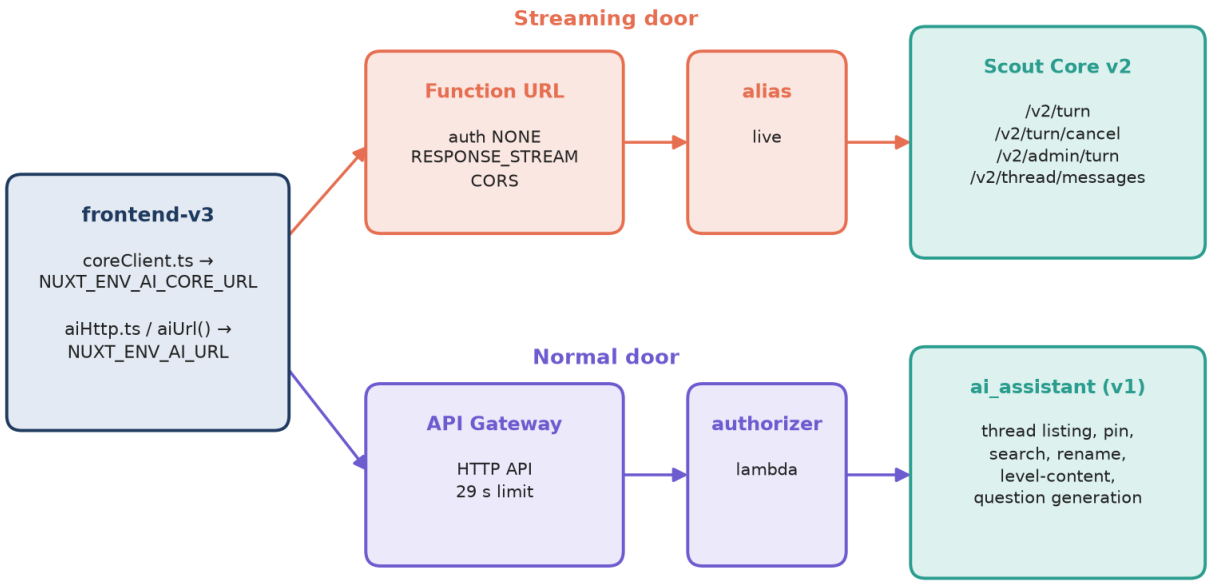


Figure 10.1 — Two doors into rag: the streaming Function URL for chat turns, API Gateway for everything else.

	Scout Core v2 (streaming)	ai_assistant (v1 door)
AWS function	`<project>-ai-assistant-v2-<env>`	`<project>-ai-assistant-<env>`
Reached through	Lambda Function URL, alias `live`	API Gateway (HTTP API) + authorizer lambda
Serves	`/v2/turn`, `/v2/turn/cancel`, `/v2/admin/turn`, `/v2/thread/messages`	thread list, search, rename, pin, level-content, question generation, other non-streaming calls
Who checks the token	`CoreAuthMiddleware` inside the function	the authorizer lambda
Frontend variable	`NUXT_ENV_AI_CORE_URL`	`NUXT_ENV_AI_URL`
Terraform	`infra/swo/scout_core_v2.tf` + `modules/scout-core-v2/`	`infra/swo/main.tf`

In simple words
Think of a cricket match. The scorecard at the end (API Gateway) is fine for most things. But for live commentary, ball by ball, you need a live channel. The Function URL is Scout's live commentary channel.

10.2 Inside the function

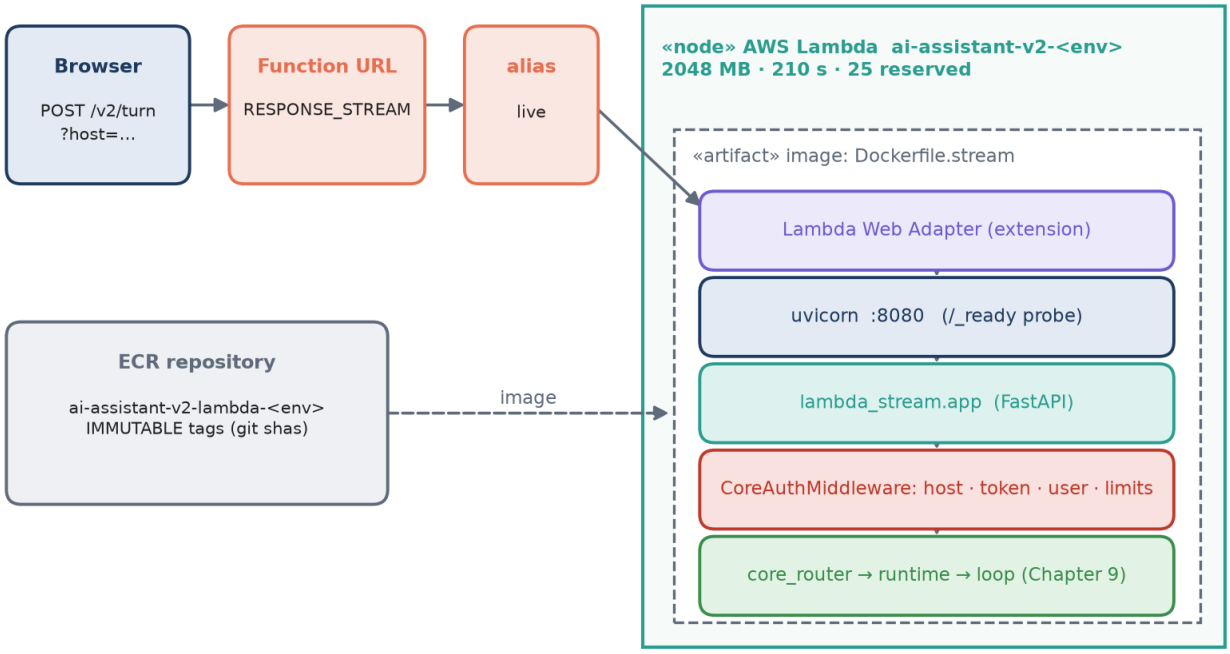


Figure 10.2 — UML deployment view of the streaming lambda.

A normal Python lambda receives one event and returns one response. Our code is a FastAPI web app that streams, so the image includes the **AWS Lambda Web Adapter**. The adapter runs as a lambda extension, starts our app with uvicorn on port 8080, waits until `/_ready` answers, and then forwards each request to the app and streams the app's response back.

`rag/lambdas/ai_assistant/Dockerfile.stream` — *final stage*

```
FROM public.ecr.aws/lambda/python:3.13
COPY --from=public.ecr.aws/awsguru/aws-lambda-adapter:1.0.1 /lambda-adapter /opt/extensions/lambda-adapter
COPY --from=builder ${LAMBDA_TASK_ROOT} ${LAMBDA_TASK_ROOT}
COPY src/ ${LAMBDA_TASK_ROOT}/src/
ARG SCOUT_GIT_SHA=unknown
ENV SCOUT_GIT_SHA=${SCOUT_GIT_SHA} \
    PYTHONPATH="${LAMBDA_TASK_ROOT}:${LAMBDA_TASK_ROOT}/src" \
    PYTHONUNBUFFERED=1 \
    AWS_LWA_INVOKE_MODE=response_stream \
    AWS_LWA_PORT=8080 \
    AWS_LWA_READINESS_CHECK_PATH=/_ready
WORKDIR ${LAMBDA_TASK_ROOT}
ENTRYPOINT ["python3", "-m", "uvicorn", "src.core.lambda_stream:app", "--host", "0.0.0.0", "--port", "8080",
            "--no-access-log"]
```

The app itself is built in `create_app()`: the readiness probe, the same `core_router` from Chapter 9, and the auth middleware wrapped around both.

rag/lambdas/ai_assistant/src/core/lambda_stream.py — *shortened*

```
def create_app(verifier=None, routers=None, spend_limit=check_spend_limits, limiter=None,
               throttle=admit) -> FastAPI:
    if routers is None:
        from src.routers.core_router import router as core_router
        routers = [core_router]
    app = FastAPI(title="Scout Core v2", docs_url=None, redoc_url=None, openapi_url=None, lifespan=_lifespan)
    app.include_router(_probe_router())
    for router in routers:
        app.include_router(router)
    app.add_middleware(CoreAuthMiddleware, verifier=verifier or CreditsVerifier(), spend_limit=spend_limit,
                       limiter=limiter, throttle=throttle)
    # Local development only. Deployed, CORS is the Function URL's configuration.
    …
    return app
app = create_app()
```

10.3 The function checks the caller itself

A Function URL has no API Gateway authorizer in front of it, and its auth type is `NONE`, so anyone on the internet can reach it. That is why `CoreAuthMiddleware` does the authorizer's job inside the function, **before a single byte of the stream is sent**:

rag/lambdas/ai_assistant/src/core/lambda_stream.py — *module docstring, shortened*

```
"""
1. The Studies Weekly host comes from `?host=` only, must match the authorizer's
   host pattern, and must be in `CORE_AUTH_ALLOWED_HOSTS`. …
2. The caller's Authorization header is checked against Laravel's credits
   endpoint, exactly as `lambdas/authorizer` does: `user_id` names the caller and
   Laravel's 404 (the assistant flag is off for them) is refused. …
3. The body's `user_id` is filled from the token, and a different one is refused,
   because on a public URL the body is whatever the caller sent.
4. The daily token limit, the in-flight limit and the per-teacher throttle
   (`utils/throttle.py`) are checked, in that order.

Every failure is plain JSON with a status, so the browser's existing error UI
applies.
"""
```

Watch out

Never trust the request body on this lambda. Step 3 exists because on a public URL the body is whatever the caller sent. The user id always comes from the verified token.

10.4 The Terraform

The streaming lambda is its own Terraform module. The two parts that matter most are the `live` alias and the Function URL attached to it.

rag/infra/swo/modules/scout-core-v2/main.tf — *shortened*

```
resource "aws_lambda_alias" "live" {
  name             = "live"
  description      = "What the Function URL serves. Moved by scripts/release-core-v2.sh; moving it back is the rollb
  function_name    = aws_lambda_function.this.function_name
  function_version = aws_lambda_function.this.version
  lifecycle {
    ignore_changes = [function_version]
  }
}
```

rag/infra/swo/modules/scout-core-v2/main.tf — *shortened*

```
resource "aws_lambda_function_url" "live" {
  function_name      = aws_lambda_function.this.function_name
  qualifier          = aws_lambda_alias.live.name
  authorization_type = "NONE"
  invoke_mode        = "RESPONSE_STREAM"
  # CORS lives here only. A function that also sends them produces duplicate
  # Access-Control-Allow-Origin values on POST, which browsers reject.
  cors {
    allow_origins  = var.cors_allowed_origins
    allow_methods  = ["GET", "POST"]
    allow_headers  = ["authorization", "content-type", "x-swo-synthetic"]
    expose_headers = ["x-scout-core-sha"]
    max_age        = 300
  }
}
```

Setting	Value	Why
Memory	2048 MB	module default (`memory_size`)
Timeout	210 s	module default; must be 120–900
`CORE_TURN_CEILING_S`	timeout − 30 = 180 s	a long turn ends with its own `error` event and a saved thread, instead of being killed by Lambda
Reserved concurrency	25	caps how many turns can run at once
Main / fast model	Claude Sonnet 5 / Claude Haiku 4.5	`scout_core_v2_model_main / _fast`
Guardrail	same Bedrock guardrail as v1	passed in from `module.bedrock_guardrails`
Image tags	git SHAs, ECR repo is IMMUTABLE	a tag always means one build

Watch out

`AWS_DEFAULT_REGION` must **not** be set in the function's environment. It is a reserved Lambda key and `CreateFunction` rejects it. The Lambda runtime sets it already. (This is the opposite of local development, where you must set it yourself.)

10.5 How the browser finds it

swo-3.0/frontend-v3/ai/services/coreClient.ts — *shortened*

```
/**
 * Where v2 lives: `NUXT_ENV_AI_CORE_URL` when set, else the assistant root.
 *
 * Deployed, the turn streams from a Lambda Function URL because the API Gateway
 * in front of the thread endpoints cannot stream, so the two roots differ.
 * Locally both are `:8002`, and a blank value keeps that. Only the turn resolves
 * here; listing, pin, search and rename stay on the assistant root.
 */
export function coreRoot(): string {
    let raw = ''
    try {
        raw = String(useRuntimeConfig().public.NUXT_ENV_AI_CORE_URL ?? '').trim()
    } catch { /* outside a Nuxt context: treated as unset */ }
    if (/^https?:\/\/\S+$/i.test(raw)) return raw.replace(/\/+$/, '')
```

swo-3.0/frontend-v3/ai/services/coreClient.ts — *shortened*

```
    const assistantRoot = aiUrl('assistant')
    if (!/^https?:\/\/(localhost|127\.0\.0\.1|\[::1\])(:|\/|$)/i.test(assistantRoot)) {
        throw new Error('AI is not configured: set NUXT_ENV_AI_CORE_URL. See .env.example.')
    }
    return assistantRoot
}
```

10.6 What the stream sends

Each event is `event: <name>` followed by `data: <json>`. The stream always ends with exactly one `turn.end` or `error`.

Event	Meaning
`turn.start`	First event (within about 300 ms): thread id, turn id, model
`status`	A teacher-facing label, for example "Reading the article"
`text.delta`	The next piece of the reply text
`artifact.start / .delta / .end`	A document (lesson plan, rubric, question set…) streamed into a card
`content_results`	Search result cards
`pending_action / pending_prompt`	The turn pauses for a browser action or a question
`turn.end`	Finished or paused, with token usage, timings and `cost_usd`
`error`	A teacher-safe message and whether retrying may help

Stop button: the browser aborts the stream *and* sends `POST /v2/turn/cancel`. On Lambda, a closed connection is not reliably visible, and the cancel may even land in a different execution environment, so the explicit cancel is the main way to stop a turn.

10.7 Deploying and rolling back

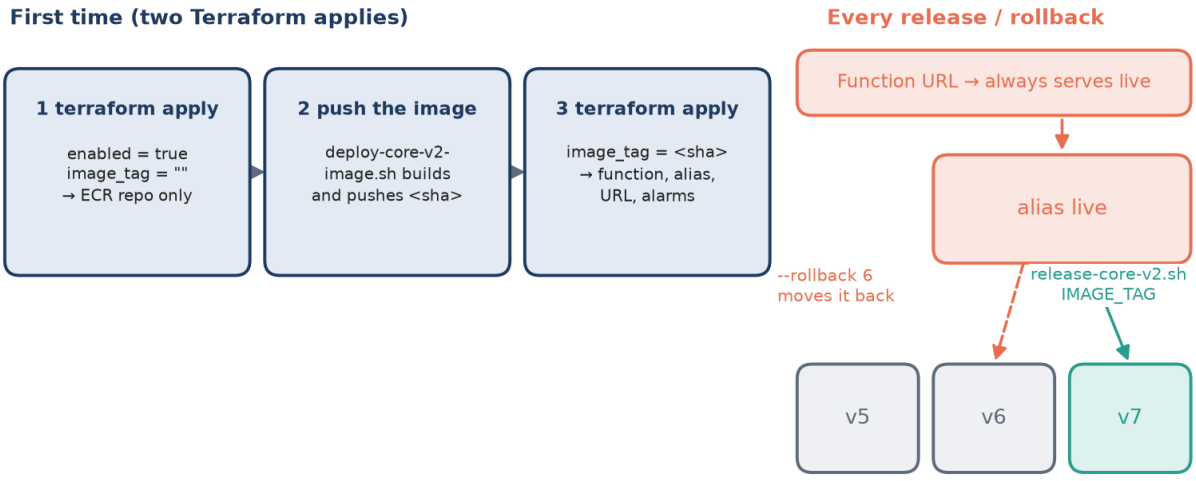


Figure 10.3 — First-time setup, then every release, then a rollback.

rag/infra/swo/scout_core_v2.tf + scripts/release-core-v2.sh — *from their header comments*

```
# First time only — two applies, because the function needs an image first:
#   1. scout_core_v2_enabled = true, scout_core_v2_image_tag = ""   → ECR repository only
#   2. infra/swo/scripts/deploy-core-v2-image.sh                     → pushes <git sha>
#   3. scout_core_v2_image_tag = "<git sha>"                         → function, alias, URL, alarms

# Every release: publish the image and move `live` to it
./scripts/release-core-v2.sh ENV PROFILE REGION IMAGE_TAG

# Rollback: move `live` back to an older version
./scripts/release-core-v2.sh ENV PROFILE REGION --rollback VERSION
```

Why it is built this way

The Function URL is attached to the `live` alias, not to a fixed version. So a release is just "move the alias", and a rollback is "move it back" — seconds, with no rebuild. Terraform ignores the alias version and the image after the first apply, so a later `terraform plan` never undoes a release. `scripts/core-v2-canary.py` is the smoke check, and the deploy runbook is `swo-3.0/docs/ScoutCoreV2Deploy.md`.

Watch out

These scripts touch shared AWS environments. On this team, deploys and Terraform applies are run only by the person who owns them. Ask before running any of them.

Chapter summary

- ✓ API Gateway cannot stream and stops at 29 s, so chat turns run in a separate lambda: Scout Core v2.
- ✓ It is reached through a Lambda Function URL (RESPONSE_STREAM, auth NONE) on the live alias.
- ✓ Lambda Web Adapter runs our FastAPI app (lambda_stream.app) with uvicorn on port 8080 inside the image.
- ✓ CoreAuthMiddleware checks host, token, user id and limits inside the function before streaming.
- ✓ The browser finds it through NUXT_ENV_AI_CORE_URL; other AI calls still use NUXT_ENV_AI_URL.
- ✓ Releases and rollbacks move the live alias; ask the owner before running them.

Chapter 11

Laravel's Part: Actions, Facts and Credits

What the backend does for Scout, and how a confirmed action works.

11.1 Three jobs

- **Confirmed actions** — create, assign, move, schedule — with preview, confirm, audit and undo.
- **Facts** — student insights, pacing, standards, misconceptions, admin figures. Numbers are computed **in PHP** from real data. The model only explains them.
- **Credits and audit** — `api/ai/user/credits` (used by the authorizer), `api/ai/stats`, feedback reports and the action history.

11.2 The action handler interface

Every server action has a handler class in `backend/app/Services/ScoutActions/Handlers/`. All of them implement the same interface:

swo-3.0/backend/app/Services/ScoutActions/ScoutActionHandler.php — *comments added*

```
interface ScoutActionHandler
{
    public function rules(): array;                                    // validation rules
    public function authorize(User $teacher, array $params): void;   // may this teacher do it?
    public function preview(User $teacher, array $params): array;    // what will change

    public function execute(User $teacher, array $params): array;    // do it
    public function authorizeUndo(User $teacher, AiAction $action): void;
    public function undoBlocker(AiAction $action): ?string;           // why undo is not possible

    public function undo(AiAction $action): void;
}
```

Examples: `AssignHandler`, `StudentsMoveHandler`, `ReplanHandler`, `CalendarScheduleHandler`, `ReteachAssignHandler`, `ClassFeedbackSaveHandler`.

11.3 The life of an action

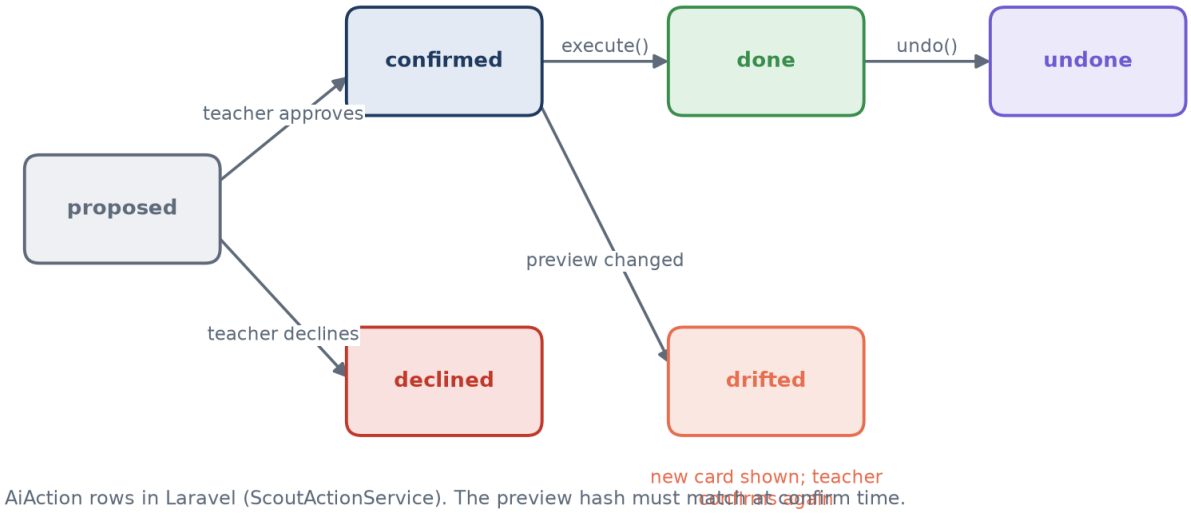


Figure 11.1 — UML state diagram of an AiAction row.

When the teacher presses **Approve**, `ScoutActionService::confirm()` runs. Read it carefully — it shows three important safety ideas.

swo-3.0/backend/app/Services/ScoutActions/ScoutActionService.php — *shortened*

```
public function confirm(User $teacher, AiAction $action, string $previewHash, string $token): AiAction
{
    $this->assertOpen($action);
    return DB::transaction(function () use ($teacher, $action, $previewHash, $token) {
        $action = AiAction::whereKey($action->id)->lockForUpdate()->firstOrFail();
        $this->assertOpen($action);
        if (!$action->confirm_token_hash
            || !hash_equals($action->confirm_token_hash, hash('sha256', $token))) {
            abort(403, 'This card can only be confirmed from the Scout window that showed it.');
        }
        …
        $handler = $this->registry->handler($action->type);
        $preview = $handler->preview($teacher, $action->params);
        $currentHash = $this->hash($action->params, $preview);
        if (!hash_equals($action->preview_hash, $previewHash)
            || !hash_equals($action->preview_hash, $currentHash)) {
            $action->update(['preview' => $preview, 'preview_hash' => $currentHash]);
            throw new ScoutActionDrifted($action);
        }
        [$result, $undoState] = $handler->execute($teacher, $action->params);
        $this->markConfirmed($action, AiActionStatus::DONE, $result, $undoState);
        return $action;
    });
}
```

- **Row lock + transaction**: two clicks cannot confirm the same action twice.
- **Confirm token**: only the Scout window that showed the card can confirm it.
- **Preview hash**: if the data changed since the teacher saw the card (for example a student was already moved), the action is *drifted*. A new card is shown and the teacher must confirm again.

11.4 The AI routes

swo-3.0/backend/routes/api.php — *shortened*

```
Route::group(['prefix' => 'ai'], function ($router) {
    Route::get('user/credits', [AiController::class, 'getUserCredits']);
    Route::post('/stats', [AiController::class, 'storeStat']);
    Route::get('/actions', [AiController::class, 'actionHistory']);
    …
        Route::get('log', [AiActionController::class, 'index']);
        Route::post('preview', [AiActionController::class, 'preview'])->middleware('throttle:30,1');
        Route::patch('{aiAction}', [AiActionController::class, 'update']);
        Route::post('{aiAction}/confirm', [AiActionController::class, 'confirm']);
        Route::post('{aiAction}/decline', [AiActionController::class, 'decline']);
        Route::post('{aiAction}/result', [AiActionController::class, 'result']);
        Route::post('{aiAction}/undo', [AiActionController::class, 'undo']);
    …
});
```

11.5 Facts are computed in PHP

Service folder	Answers questions like…
`Services/StudentInsights/`	Who needs help? Missing work? Standards coverage? A student's profile?
`Services/Pacing/`	Where is my class compared with where it should be?
`Services/Misconceptions/`	Which wrong answers were common? Which skill groups need reteaching?
`Services/Differentiation/`	Suggested groups and matched leveled readers
`Services/AdminInsights/`	District and school figures for admins
`Services/PlanningCalendar/`	The class's yearly calendar overview

Why it is built this way

Models count badly. So every number a teacher or admin sees is computed by our code from real data, with its formula and population known. Scout's job is to explain the number, never to invent it.

Worked example: "Where is my class compared with where it should be?"

Let us follow this one question through both repos. The important idea: the model **never works out a week itself**. The constitution tells it that a class's position comes only from the `pacing.where` action, so it asks the browser to run that action, and the browser asks Laravel.

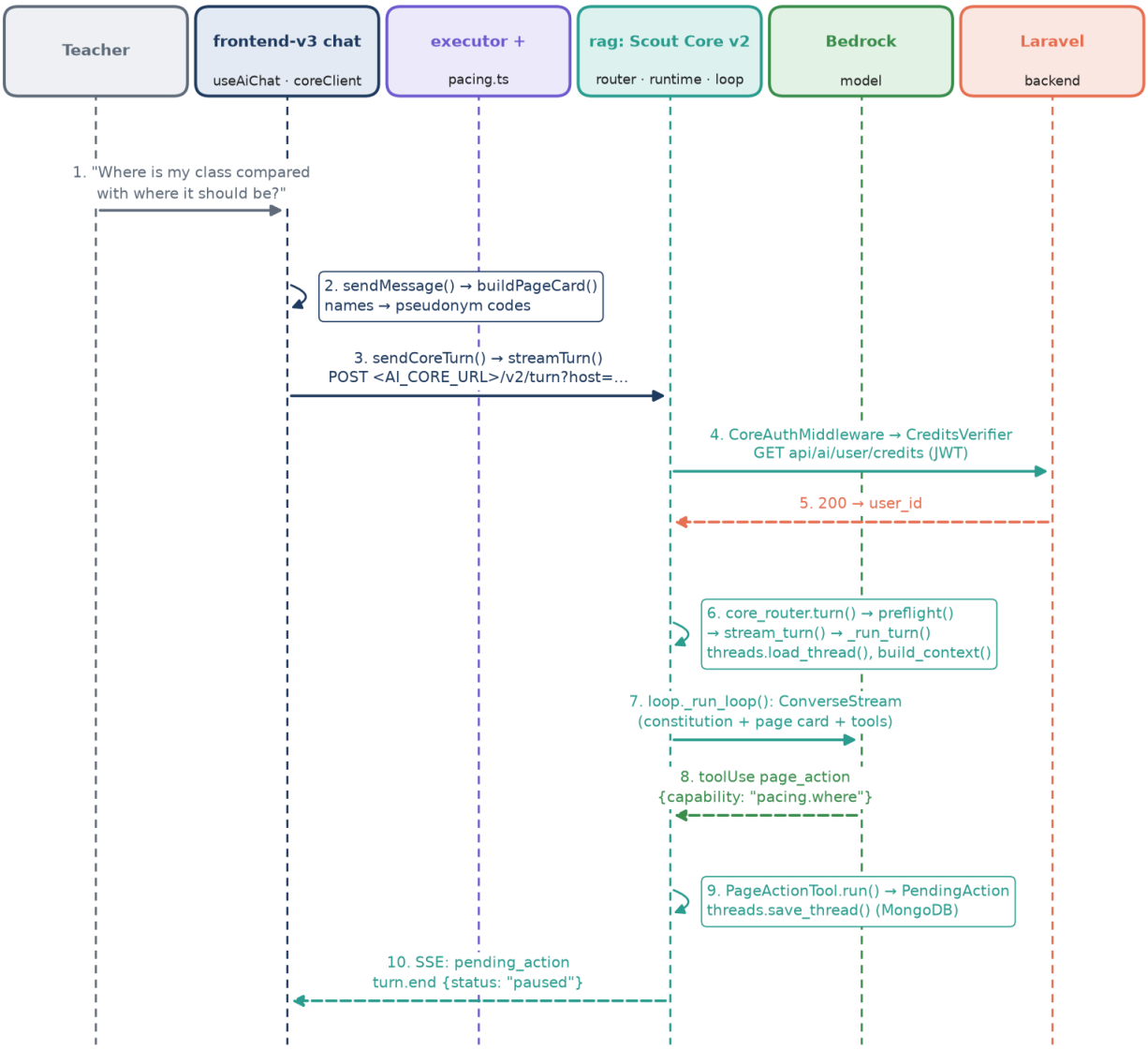


Figure 11.2 — Part 1: the question reaches rag, and the model asks for pacing.where (the turn pauses).

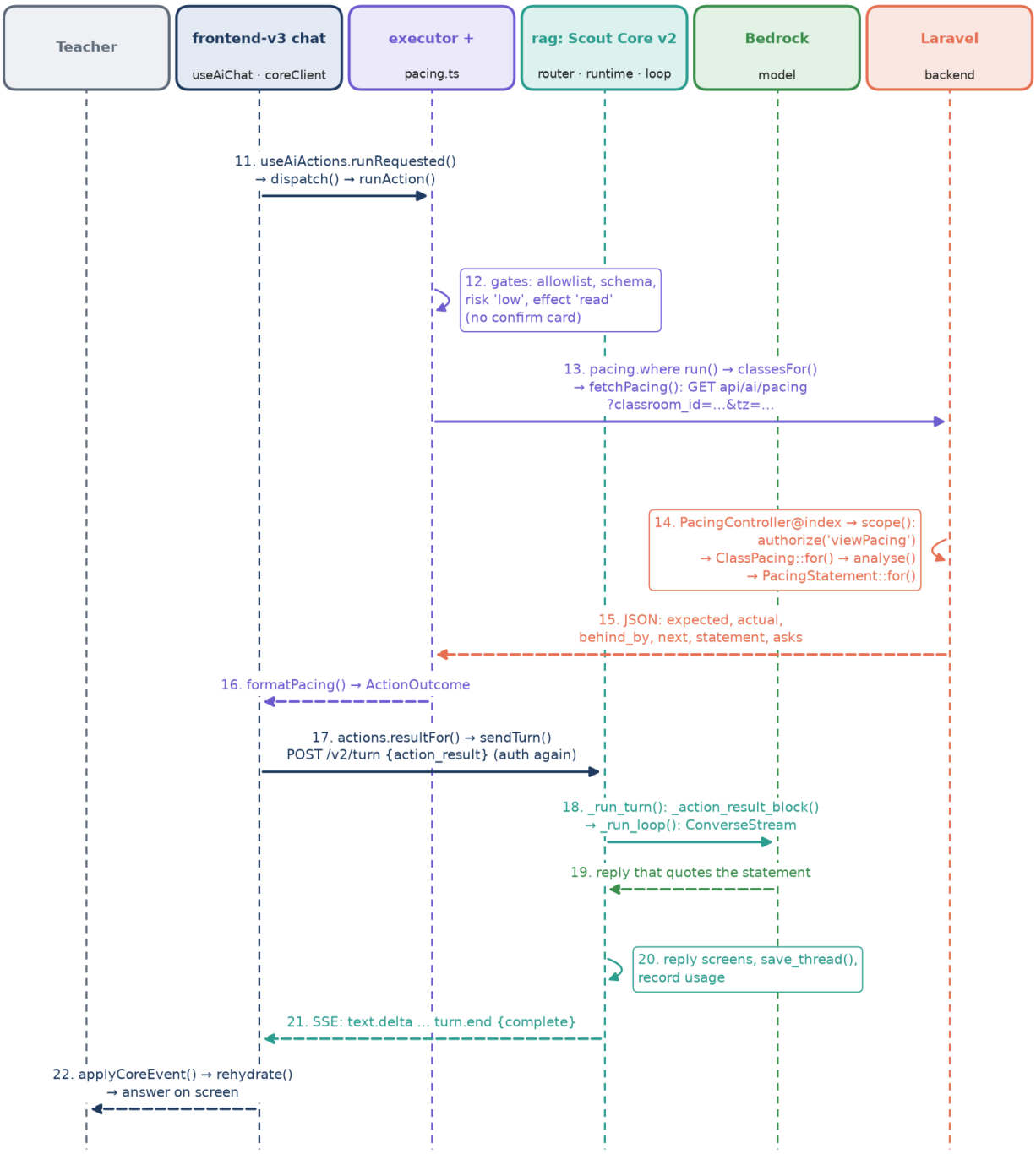


Figure 11.3 — Part 2: the browser gets the figures from Laravel, and the turn resumes so the model can quote them.

Some things to notice in the diagrams:

- **Steps 4–5:** the streaming lambda checks the teacher's token with Laravel before anything else (Chapter 10). Step 17 is a new request, so it is checked again.
- **Steps 8–10:** the model's tool call does not run anything in rag. `PageActionTool.run()` only packs a `PendingAction`, saves the thread and pauses the turn.
- **Step 12:** `pacing.where` is `risk: 'low'` and `effect: 'read'`, so it runs at once with no confirmation card. It changes nothing.
- **Steps 13–15:** the browser calls Laravel with the teacher's own login, so Laravel's `viewPacing` policy decides what the teacher may see. With no class named, it uses the class on the page,

or else every class the teacher teaches (`classesFor()`).

- **Step 14:** `ClassPacing::analyse()` compares the *expected* week (from the class calendar or planning calendar) with the *actual* week (from finished work), and `PacingStatement::for()` writes the sentence the model must quote.
- **Step 19:** the model quotes the statement; it does not do the maths. If Laravel returned `asks` (for example the class's first day), the model asks the teacher that question.

swo-3.0/frontend-v3/ai/composables/useScoutActions.ts — *step 13*

```
const fetchPacing = async (classroomId: string, publicationId?: number, calendar?: Record<string, unknown>) => {
    try {
        const params = {
            classroom_id: classroomId,
            tz: timeZone(),
            ...(publicationId ? { publication_id: publicationId } : {}),
            ...calendar,
        }
        const { data } = await $axios.get('ai/pacing', { params, ...quiet })
        return data.data
    } catch (error) {
        return refused(error)
    }
}
```

swo-3.0/backend/app/Http/Controllers/PacingController.php — *step 14, shortened*

```
public function index(PacingRequest $request, ClassPacing $pacing): JsonResponse
{
    [$classroom, $publications] = $this->scope($request, $request->filled('publication_id')
        ? [(int) $request->input('publication_id')]
        : null);
    $today = $this->today($request->input('tz'));
    $calendar = $this->calendarOverride($request);

    return response()->json([
        'data' => $publications
            ->map(fn (Publication $publication) => $pacing->for(
                $classroom, $publication, $request->user(), $today, $calendar
            ))
            ->values(),
    ]);
}

private function scope(PacingRequest $request, ?array $publicationIds): array
{
    $classroom = Classroom::findOrFail($request->input('classroom_id'));
    $this->authorize('viewPacing', $classroom);
    …
}
```

Rule for every new or changed endpoint: write a feature test, and run `vendor/bin/phpcs -n` on the files you changed (PSR-12).

Chapter summary

- ✓ Laravel does three jobs for Scout: confirmed actions, computed facts, and credits/audit.
- ✓ Each server action is a ScoutActionHandler with rules, authorize, preview, execute and undo.
- ✓ confirm() uses a row lock, a confirm token and a preview hash; changed data means 'drifted', not done.
- ✓ All numbers are computed in PHP services; the model only explains them.
- ✓ A pacing question runs: rag pauses → browser runs pacing.where → Laravel ClassPacing → rag resumes and quotes.
- ✓ New endpoints need feature tests and phpcs.

Chapter 12

Cost, Speed and Safety

How we chose the model, how we cap spending, and how we keep replies safe.

12.1 Choosing the model

In ONLINE-6859 (August 2026) we measured real Bedrock calls for the **original v1 assistant**, instead of guessing. Nova Micro picked the right action almost as often as larger models, at a small fraction of the cost.

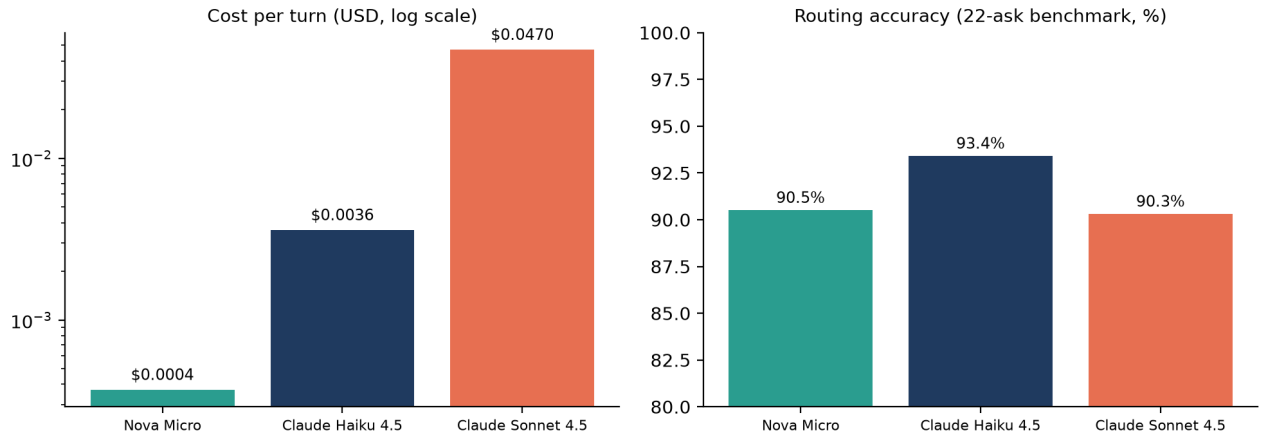


Figure 12.1 — Measured cost per turn and routing accuracy (docs/AiAssistantModelEvaluation.md).

Watch out

This study was about v1. **Scout Core v2 uses Claude models:** `us.anthropic.claude-sonnet-5` as the main model and Claude Haiku 4.5 as the fast model (for titles and quick checks). They are set in `core/config.py` and in Terraform (`scout_core_v2_model_main` / `scout_core_v2_model_fast`). In the ONLINE-7526 check on dev, a v2 turn cost about **$0.023** on average.

In simple words

The study's lasting lesson: invented links were fixed in code, not by a bigger model. That is still true for v2.

12.2 What each feature costs

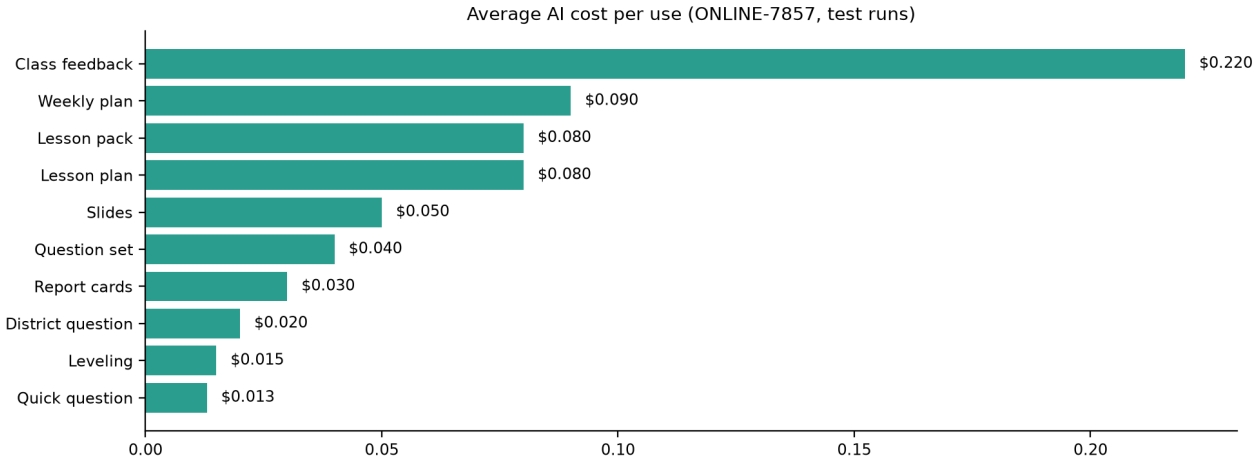


Figure 12.2 — Average AI cost per use of each Scout tool.

12.3 The spending caps

Every teacher has a **$5 daily** cap and a **$2.50 weekly** cap (staff get $10 a week). The cap counts real AI cost, so a lesson pack uses more than a quick question. Usage is stored in DynamoDB.

rag/lambdas/ai_assistant/src/services/daily_usage_service.py — *shortened*

```
DAILY_COST_LIMIT_USD = 5.0
WEEKLY_COST_LIMIT_USD = 2.5
STAFF_WEEKLY_COST_LIMIT_USD = 10.0
def check_spend_limits(self, user_id: str, now: Optional[datetime] = None,
                       raised_spend_cap: bool = False) -> Dict[str, Any]:
    user_id = str(user_id)
    now = now or datetime.now(UTC)
    today = now.strftime("%Y-%m-%d")
    spent = self._spend_this_week(user_id, now)
    day_micro, week_micro = spent.get(today, 0), sum(spent.values())
    weekly_limit_usd = STAFF_WEEKLY_COST_LIMIT_USD if raised_spend_cap else self.weekly_cost_limit_usd
    day_limit, week_limit = _micro(self.daily_cost_limit_usd), _micro(weekly_limit_usd)
    code = WEEKLY_CODE if week_micro >= week_limit else DAILY_CODE if day_micro >= day_limit else None
    …
    if code is not None:
        period = "this week's" if code == WEEKLY_CODE else "today's"
        message = f"You've reached {period} limit for Scout. You can continue in {_wait(reset - now)}."
    return {"allowed": code is None, "code": code, …, "message": message}
```

Money is stored as whole **micro-dollars** (`_micro()`) so rounding never adds up wrongly.

12.4 Making turns cheaper and faster

Three tickets made each turn cheaper and faster: ONLINE-7526, ONLINE-7528 and ONLINE-7588. All three are about the same idea, **prompt caching**, so let us understand that first.

First, how prompt caching works

Every call to the model sends a long request: the tool descriptions, Scout's rules, the page card, the conversation so far, and the new message. One turn can call the model several times (once before each tool and once after), and most of that request is the same every time.

Bedrock can remember the **beginning** of a request. We put a marker called a `cachePoint` in the request. If the next request starts with **exactly the same bytes** up to that marker, Bedrock reads that part from its cache instead of processing it again. Cached reading is much cheaper and faster than fresh input. In our code the tools and the shared core are cached for one hour, and other cache points for five minutes.

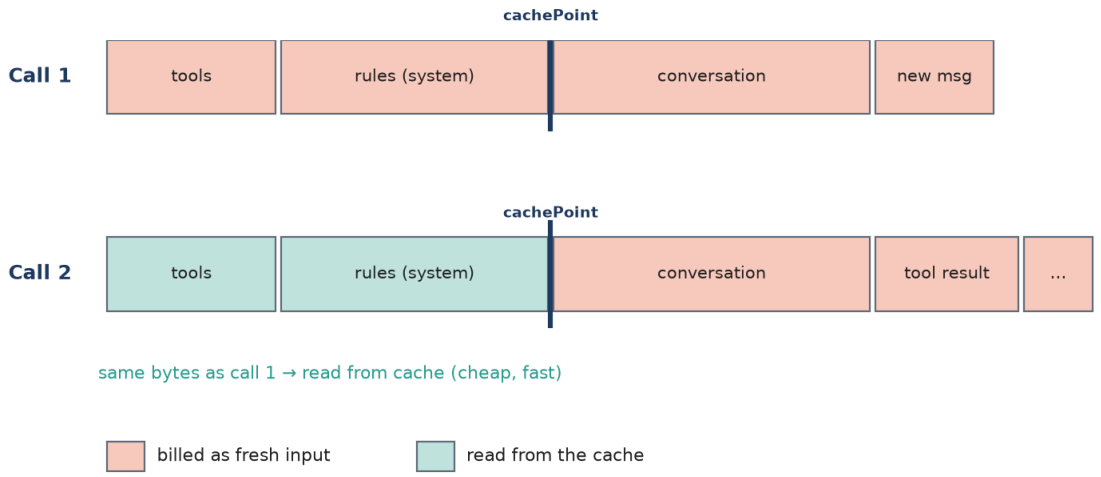


Figure 12.3 — Call 2 starts with the same bytes as call 1, so that part is read from the cache.

In simple words
Like a tailor who already has your measurements on file. On your second visit they only need to hear what is new. But if even one measurement changes at the top of the card, they must take all the measurements again.

Watch out
The golden rule of caching: **anything that changes must come after the cache point**. If one byte changes near the start, everything after it is billed fresh again.

ONLINE-7526 — cache the conversation, cap spending in dollars

	What it means
The problem	A typical turn processed about 12,300 tokens. Caching stopped after the tools and the rules, so on every tool step the whole conversation history was billed again as fresh input. Also, the daily cap counted only uncached tokens, so it hardly limited real spending.
What we changed	First we measured real deployed turns from CloudWatch (timings per stage, cache hits, cost per turn). Then we added a second `cachePoint` at the end of the history, wrote one usage record per turn that includes cached tokens, made the daily cap a dollar amount ($5), and checked what the safety guardrail itself costs.

	What it means
The result	In the QA check on dev, cache reads were 82% of all input-side tokens and only 2% was billed as fresh input. The average turn cost about $0.023, and the $5 cap now counts real cost.

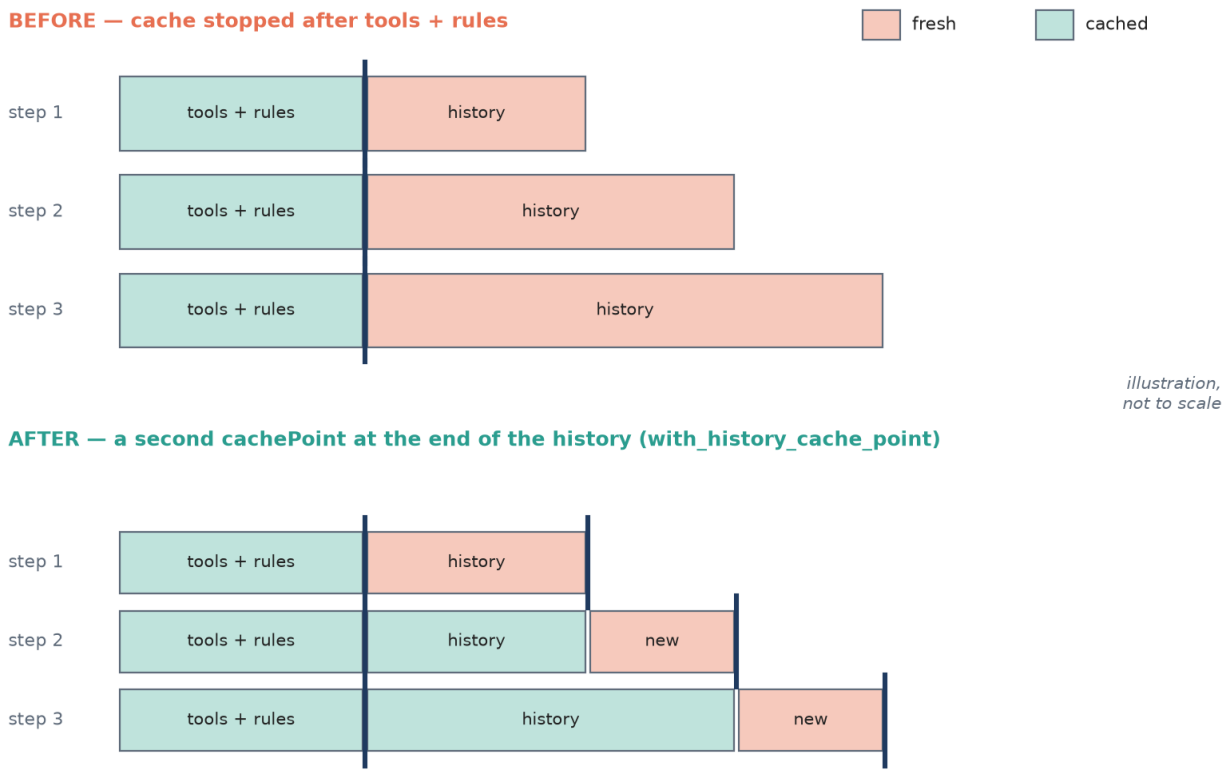


Figure 12.4 — Before, the history was re-billed on every step. After, each step pays only for what is new.

rag/lambdas/ai_assistant/src/core/loop.py

```
def with_history_cache_point(messages: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """A copy whose last message ends with a `cachePoint`; the stored history never carries one.
    The next call (a tool step, or the next turn within the cache's five minutes) sends the same
    messages plus a few more, and Bedrock's lookback finds this checkpoint within 20 blocks, so the
    conversation so far is read at the cache-read rate instead of re-billed as fresh input.
    """
    if not messages:
        return messages
    last = messages[-1]
    content = list(last.get("content") or [])
    if not content or "cachePoint" in content[-1]:
        return messages
    return [*messages[:-1], {**last, "content": [*content, CACHE_POINT]}]
```

Notice: the cache point is added to a **copy**. The history saved in MongoDB never carries it, and Bedrock allows at most four cache points in one request (`MAX_CACHE_POINTS = 4`).

ONLINE-7528 — page switches stop re-paying the cache

	What it means
The problem	The list of actions on the current page was written inside the `page_action` tool definition, as a list of allowed ids. Tool definitions sit at the very start of the request. So when the teacher moved to another page, the first block changed and about **9,000 cached tokens** had to be written again. After ONLINE-7526, this was the biggest waste left.
What we changed	The `page_action` tool now looks the same on every page. The page's list of actions moved into the page context, which comes after the cached part. Nothing Scout can do on a page changed.
The result	Switching pages no longer throws away the cached tools and rules. Only the small page part is new.

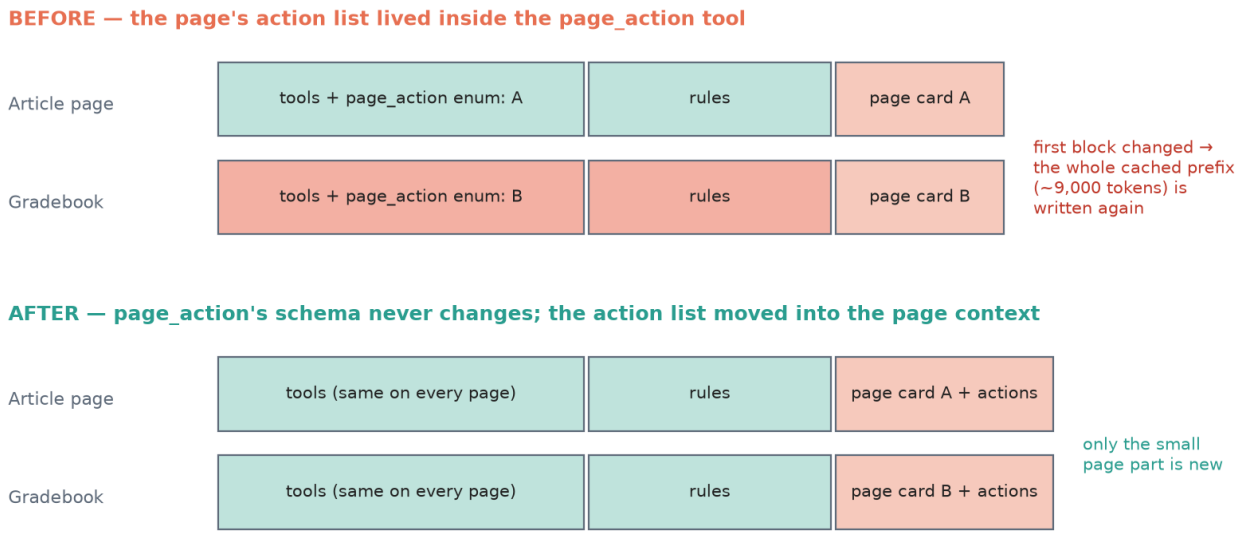


Figure 12.5 — Before, a page switch changed the first block and broke the whole cache. After, only the page part changes.

rag/lambdas/ai_assistant/src/core/tools/page_action.py — *shortened*

```
def _spec() -> Dict[str, Any]:
    # ONLINE-7528: no per-page enum here. The page's capability ids are already listed, with their
    # args and descriptions, in the page-context system block (context/build.py: render_capabilities);
    # repeating them as an enum bought nothing but a schema that changed on every page switch, which
    # rewrote the whole tools cache prefix (~9k tokens) each time. `run()` below is still the one place
    # that enforces which ids this turn may act on.
    …

# inside PageActionTool.run():
if not capability or (ctx.capability_ids and capability not in ctx.capability_ids):
    return ToolOutcome.error(
        f"'{capability}' is not an action this page offers. Choose one of the "
        "capability ids in the page context, or answer in words if none fits."
    )
```

Why it is built this way

Moving the list did not make Scout less safe. The check moved from the tool's schema into `run()`, which still refuses any action the page does not offer, and the browser's executor checks the allowlist again (Chapter 6).

ONLINE-7588 — a small shared core; rules travel with the feature

	What it means
The problem	Every chat call carried about **17,700** identical tokens: about 8,800 of rules for every feature and about 8,600 of descriptions for 18 tools. Each new feature made every call bigger, even for teachers who never used it.
What we changed	A shared core of about **5,200** tokens keeps only the rules that must always apply. Tool descriptions now say only *when* to use the tool. Tool results carry facts, never rules. Each page's module loads in a fixed, predictable way, and the teacher's "About you" block has its own cached layer. A rule registry gives every rule sentence exactly one home, and a test caps the size of the core plus tools.
How it was checked	Quality came first. The quality gate was fixed before any test ran: 300 frozen questions asked twice on each side, plus 40 conversations, old version against new. The change could merge only if quality was no worse; cost and speed were reported only after that.

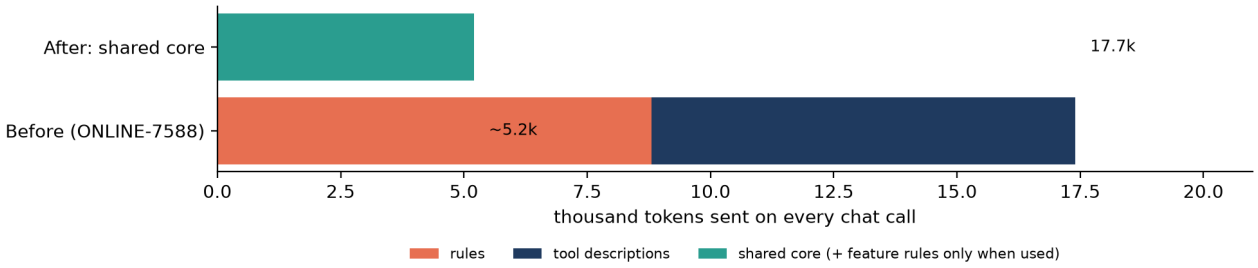


Figure 12.6 — Tokens sent on every chat call, before and after the prompt restructure.

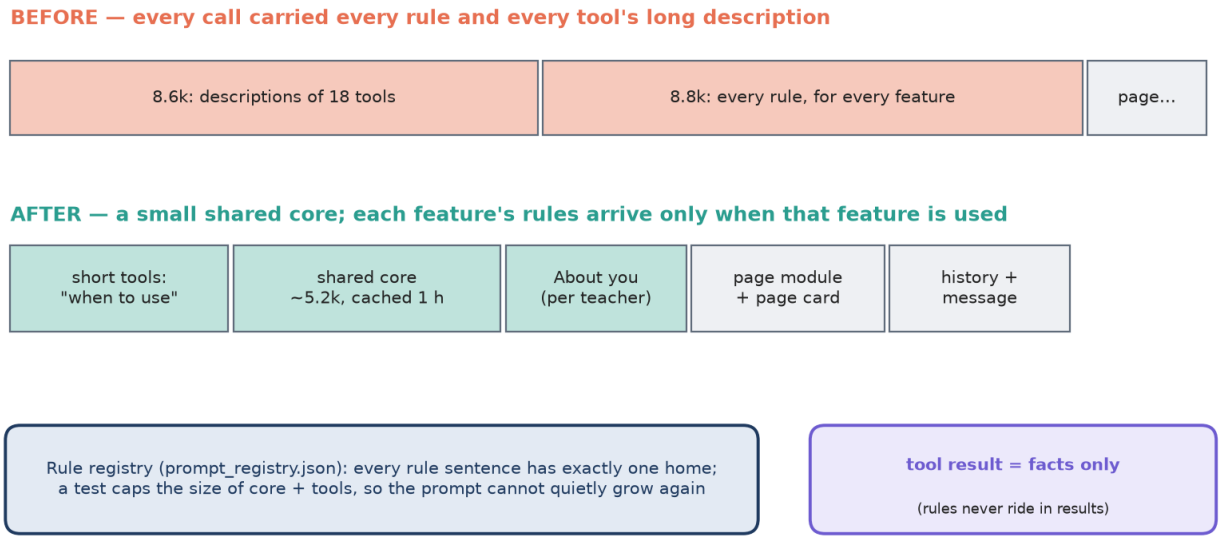


Figure 12.7 — The request before and after: a small cached core, with each feature's rules arriving only when needed.

In simple words

Before, Scout carried the whole school library in its bag every day. Now it carries one small notebook of rules it always needs, and picks up a book only for the lesson it is teaching.

Ticket	In one line
ONLINE-7526	Cache the conversation too, and count the daily cap in real dollars.
ONLINE-7528	Keep the tools identical on every page, so a page switch keeps the cache.
ONLINE-7588	Shrink what every call carries; load feature rules only when the feature is used.

12.5 Safety

- The **Bedrock guardrail** blocks harmful requests. ONLINE-7529 made sure it runs on the tier Terraform declares; on that tier it blocked all 60 harmful test requests (before the fix, 3 got through).
- The **router checks** message and page size, identity and thread ownership before the model runs.
- The **screens** (Chapter 9) check every reply.

Chapter summary

✓ v1 was measured on Nova Micro; Scout Core v2 runs Claude Sonnet 5 (main) and Haiku 4.5 (fast), about $0.023 per turn on dev.

✓ Teachers have $5/day and $2.50/week caps, counted in real AI cost and stored as micro-dollars.

✓ Prompt caching, stable tool lists and a smaller shared prompt cut cost and time.

✓ Safety comes from the guardrail, early router checks and reply screens.

Chapter 13

Testing and Checking Your Work

Which check to run for which change.

13.1 The quick guide

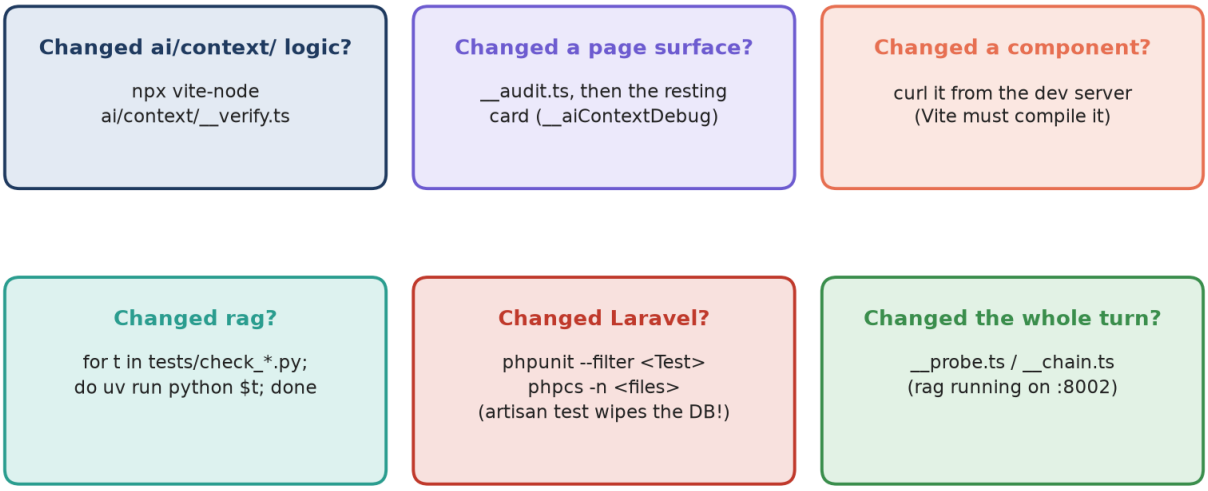


Figure 13.1 — Pick the check that matches what you changed.

13.2 frontend-v3: executable specs

frontend-v3 has **no unit test runner** and no `vue-tsc`. Instead, pure logic in `ai/context/` is covered by hand-written executable specs that run with vite-node.

swo-3.0/frontend-v3/ai/context/__verify.ts — *header*

```
/**
 * Executable specification for the capability registry and executor.
 *
 *     npx vite-node ai/context/__verify.ts
 *
 * `frontend-v3` has no test runner, and this layer is the one place where a
 * silent regression would let the assistant report success for something that
 * did not happen. Written against no framework so it stays runnable whatever
 * the project adopts later.
 *
 * Not imported by anything, so it never reaches the bundle.
 */
```

terminal

```
cd ~/swo-3.0/frontend-v3
npx vite-node ai/context/__verify.ts         # ~2,800 assertions
npx vite-node ai/context/__audit.ts          # every page against the budgets
npx vite-node ai/context/__probe.ts [page]   # real page card → live assistant on :8002
npx vite-node ai/context/__chain.ts [runs]   # a whole turn, end to end
# Does a component compile? Ask the running dev server to transform it:
curl -sk -o /dev/null -w "%{http_code}\n" "https://localhost:3002/beta/_nuxt/<abs-path-to-file>"
```

Why it is built this way
Put pure logic in its own module under `ai/context/` (like `roster.ts`, `gradebookStats.ts`, `outline.ts`), and keep only the Vue-specific half in the component. Then `__verify.ts` can test it without a browser.

13.3 rag: check scripts

terminal

```
cd ~/rag/lambdas/ai_assistant
for t in tests/check_*.py; do uv run python "$t" || break; done
uv run python tests/check_router.py          # just one
```

They are named `check_*.py` (not `test_*.py`) so pytest skips them. Each one runs on import, prints PASS or FAIL, and exits non-zero on failure. If your script uses a live MongoDB, give its thread ids a per-run suffix, or runs will resume each other's conversations.

13.4 Laravel

terminal

```
docker exec -w /online-api online-api.studiesw.test php vendor/bin/phpunit --filter AiActionControllerTest
docker exec online-api.studiesw.test vendor/bin/phpcs -n <changed files>
```

Watch out
Running the full `php artisan test` wipes your local database (52 tests use RefreshDatabase). Use `--filter`. To recover: `php artisan migrate:fresh --seed`.

Chapter summary

- ✓ frontend-v3 logic is tested with vite-node specs like __verify.ts (no vitest, no vue-tsc).
- ✓ Check a component compiles by fetching it from the running dev server.
- ✓ rag uses check_*.py scripts that print PASS/FAIL; give live-Mongo threads a unique suffix.
- ✓ Laravel: phpunit with --filter, phpcs on changed files; never the full suite on your dev DB.

Chapter 14

Your First Feature: A Recipe

Step by step, from a ticket to a working Scout ability.

Let us say your ticket is: "*Teachers on the People page can ask Scout to show only students who have audio reader turned on.*" Here is how you would build it.

14.1 The steps

#	Step	Where
1	Read the ticket and the docs: StartHere §2 rules, the Contract, PageCoverage for this route.	`swo-3.0/docs/`
2	Find the component that already knows the data (the students list).	`components/classroom/people/StudentsList.vue`
3	Publish a short **view** fact: "4 of 26 students have audio reader on" — an answer, not rows.	`useAiSurface({ view })`
4	Add a **low-risk capability** whose `run()` calls the page's own filter, and whose `verify()` re-reads the filter state.	`capabilities: [ … ]`
5	Use `publicFacts()` / pseudonyms for anything about a student.	`ai/context/roster.ts`
6	If the logic is more than a few lines, move it into a pure module and add specs.	`ai/context/<name>.ts + __verify`
7	Check the resting card with `__aiContextDebug` and run `__audit.ts`.	browser + terminal
8	Only if the model picks the wrong action: improve the capability's `title` wording first. Change rag prompts only when that is not enough.	`rag/…/prompts/`
9	Run the checks from Chapter 13, then a live `__probe.ts` on the page.	terminal
10	Commit as `ONLINE-####: lowercase summary` with a body that explains why. No AI attribution on this workstream.	git

14.2 A sketch of the code

example only — not in the repo

```
// StudentsList.vue  (sketch — follow the RateArticle pattern from Chapter 5)
useAiSurface({
    name: 'People list',
    view: () => ({
        audioReaderOn: `${audioCount.value} of ${students.value.length} students`,
        listFilter: filter.value ?? 'none',
    }),
    capabilities: [
        capability({
            id: 'people.filterAudioReader',
            panelLabel: 'Show audio reader students',
            title: 'Show only students with Audio Reader turned on.',
            risk: 'low',
            run: () => setFilter('audio_reader'),
            verify: () => filter.value === 'audio_reader',
```

example only — not in the repo

```
            describeResult: () => 'Showing only students with Audio Reader on.',
        }),
    ],
})
```

14.3 Before you open a merge request

- No student name, username or password can reach rag (check the card in the console).
- Every new capability has a real `verify()`, or its result text says only what it knows.
- Numbers are computed by code, not left for the model to count.
- Nothing is cut silently; if something does not fit, the text says so.
- frontend specs, rag check scripts and Laravel tests (if touched) all pass.
- Do not refactor old code just to match the current style. New code follows current style.
- Keep `CLAUDE.md`, `AGENTS.md` and editor folders out of feature commits.

14.4 Where each feature lives

Feature area	frontend-v3 starting point	rag starting point
Page awareness	`ai/context/pageCard.ts`, `executor.ts`	`core/runtime.py`, `tools/page_action.py`
Privacy	`ai/context/roster.ts`, `studentNames.ts`	`core/name_scrub.py`, `name_claim.py`
Memory	`ai/stores/scoutMemory.ts`	`core/context/teacher.py`, `tools/remember.py`
Confirmed actions	`ai/context/scoutActions.ts`	`tools/draft_assignment.py` (+ Laravel handlers)
Search	`ai/services/contentResults.ts`	`tools/search_curriculum.py`
Leveling	`ai/context/articleLevel.ts`	`tools/level_article.py`, `engines/level.py`
Questions	`ai/intents/generateQuestionSet.ts`	`engines/questions.py`
Lesson plans	`utils/lessonPlanCompose.ts`	`engines/compose.py`, `tools/lesson_pack.py`
Insights	`ai/context/studentInsights.ts`	`prompts/constitution.py` (+ Laravel services)
Grading	`ai/context/grading.ts`	`engines/grade.py`, `engines/group.py`
Admin Scout	`components/insights/InsightsPage.vue`	`core/admin/turn.py`, `admin/tools.py`
Cost	`ai/utils/errorMessages.ts`	`services/daily_usage_service.py`

14.5 Glossary

Word	Meaning
Surface	What a mounted component publishes about itself for Scout
Capability	One typed action Scout may take, with run() and verify()
Page card	Everything the browser sends about the page on each turn
Executor	The single gate that runs and verifies every action (executor.ts)
Pseudonym / alias	A stable code like Student K7QF2M that stands for a real student
Turn	One teacher message and Scout's full reply, including tool calls
SSE	Server-Sent Events: how rag streams the reply to the browser bit by bit
Tool	Something the chat model may call (search, level, write...)
Engine	A focused model call that does one job with its own prompt
Pause / resume	A turn stops to let the browser act or ask, then continues
Screen	A check that runs on every reply before the teacher sees it
Constitution	Scout's shared, byte-stable system prompt
Drifted	An action whose preview changed before confirm; the teacher must confirm again
Bedrock	The AWS service that hosts the AI model

Chapter summary

- ✓ Start from the component that already knows the data; publish a short answer, not rows.
- ✓ Add a capability that calls the page's own function and verifies the result.
- ✓ Use pseudonyms for students, compute numbers in code, never cut silently.
- ✓ Fix wording before touching rag prompts; run the right checks; commit as ONLINE-####: summary.
- ✓ Welcome to the team — you are ready to build your first Scout feature!